



 Latest updates: <https://dl.acm.org/doi/10.1145/3603170>

RESEARCH-ARTICLE

Side-channel and Fault-injection attacks over Lattice-based Post-quantum Schemes (Kyber, Dilithium): Survey and New Results

PRASANNA RAVI, School of Computer Science and Engineering,
Singapore City, Singapore

ANUPAM CHATTOPADHYAY, Nanyang Technological University,
Singapore City, Singapore

JAN PIETER D'ANVERS, KU Leuven, Leuven, Vlaams-Brabant, Belgium

ANUBHAB BAKSI, Nanyang Technological University, Singapore City,
Singapore

Open Access Support provided by:

Nanyang Technological University

KU Leuven

School of Computer Science and Engineering



PDF Download
3603170.pdf
05 April 2026
Total Citations: 57
Total Downloads:
11025

Published: 28 March 2024

Online AM: 05 June 2023

Accepted: 07 May 2023

Revised: 28 February 2023

Received: 10 May 2022

[Citation in BibTeX format](#)

Side-channel and Fault-injection attacks over Lattice-based Post-quantum Schemes (Kyber, Dilithium): Survey and New Results

PRASANNA RAVI, School of Computer Science and Engineering, Nanyang Technological University, Singapore

ANUPAM CHATTOPADHYAY, Temasek Labs, Nanyang Technological University, Singapore

JAN PIETER D'ANVERS, imec-COSIC, KU Leuven, Belgium

ANUBHAB BAKSI, Temasek Labs, Nanyang Technological University, Singapore

In this work, we present a systematic study of Side-Channel Attacks (SCA) and Fault Injection Attacks (FIA) on structured lattice-based schemes, with main focus on Kyber Key Encapsulation Mechanism (KEM) and Dilithium signature scheme, which are leading candidates in the NIST standardization process for Post-Quantum Cryptography (PQC). Through our study, we attempt to understand the underlying similarities and differences between the existing attacks while classifying them into different categories. Given the wide variety of reported attacks, simultaneous protection against all the attacks requires to implement customized protections/countermeasures for both Kyber and Dilithium. We therefore present a range of customized countermeasures, capable of providing defenses/mitigations against existing SCA/FIA, and incorporate several SCA and FIA countermeasures within a single design of Kyber and Dilithium. Among the several countermeasures discussed in this work, we present novel countermeasures that offer simultaneous protection against several SCA- and FIA-based chosen-ciphertext attacks for Kyber KEM. We implement the presented countermeasures within two well-known public software libraries for PQC: (1) *pqm4* library for the ARM Cortex-M4-based microcontroller and (2) *liboqs* library for the Raspberry Pi 3 Model B Plus based on the ARM Cortex-A53 processor. Our performance evaluation reveals that the presented custom countermeasures incur reasonable performance overheads on both the evaluated embedded platforms. We therefore believe our work argues for usage of custom countermeasures within real-world implementations of lattice-based schemes, either in a standalone manner or as reinforcements to generic countermeasures such as masking.

CCS Concepts: • **Security and privacy** → **Side-channel analysis and countermeasures**;

Additional Key Words and Phrases: Lattice-based cryptography, side-channel attacks, fault-injection attacks, Kyber, Dilithium

This work was supported in part by CyberSecurity Research Flanders with reference number VR20192203, the Research Council KU Leuven (C16/15/058) and the Horizon 2020 ERC Advanced Grant (101020005 Belfort). Jan Pieter D'Anvers is funded by FWO (Research Foundation Flanders) as junior post-doctoral fellow (contract number 133185 / 1238822N LV).

Authors' addresses: P. Ravi and A. Baksi, Temasek Labs, Nanyang Technological University, 50 Nanyang Drive, Research Techno Plaza, BorderX Block, 9th Storey, Singapore 637553; emails: prasanna.ravi@ntu.edu.sg, anubhab.baksi@ntu.edu.sg; A. Chattopadhyay, School of Computer Science and Engineering, Nanyang Technological University, 50 Nanyang Avenue, Singapore 639798; email: anupam@ntu.edu.sg; J. P. D'Anvers: Computerbeveiliging & Ind.Crypt.Leuven, Kasteelpark Arenberg 10 - bus 2452, 3001 Heverlee, Belgium; email: janpieter.danvers@esat.kuleuven.be.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1539-9087/2024/03-ART35 \$15.00

<https://doi.org/10.1145/3603170>

ACM Reference format:

Prasanna Ravi, Anupam Chattopadhyay, Jan Pieter D’Anvers, and Anubhab Baksi. 2024. Side-channel and Fault-injection attacks over Lattice-based Post-quantum Schemes (Kyber, Dilithium): Survey and New Results. *ACM Trans. Embedd. Comput. Syst.* 23, 2, Article 35 (March 2024), 54 pages.
<https://doi.org/10.1145/3603170>

1 INTRODUCTION

In 2016, the **National Institute for Standards and Technology (NIST)** initialized a global level standardization process for *quantum attack* resistant public-key cryptographic schemes [1], which is otherwise known as **Post-Quantum Cryptography (PQC)**. Very recently in 2022, after three rounds of evaluation, NIST announced the first standards for PQC in the category of **Public Key Encryption (PKE)**, **Key Encapsulation Mechanisms (KEM)**, and **Digital Signatures (DS)** [2]. Theoretical post-quantum security guarantees and implementation performance on different HW/SW platforms served as the primary criteria for selection in the initial rounds of the NIST standardization process. However, resistance against **side-channel attacks (SCA)** and **fault injection attacks (FIA)** as well as the cost of implementing protections against SCA and FIA, also emerged as a very important criterion towards the latter part of the standardization process. This is especially true, when it comes to comparing schemes with tightly matched security and efficiency [4]. In Reference [1, Sections 3.4 and 2.2.3] NIST states that it *encourages additional research regarding side-channel analysis* of the finalist candidates and *hopes to collect more information about the costs of implementing these algorithms in a way that provides resistance to such attacks*.

Three out of the seven finalist candidates derive their hardness from the well-known **Learning With Error (LWE)** and **Learning With Rounding (LWR)** problem that operate structured lattices. In this respect, Kyber [5] and Dilithium [24], based on the LWE problem were selected as the first standards for KEMs and signature schemes, respectively. Moreover, these schemes have received considerable attention with several works demonstrating practical attacks [11, 51, 52, 63], particularly on embedded targets. These attacks have been realized using a wide-range of attack vectors such as power and Electromagnetic Emanation (EM) for SCA and voltage/clock glitching and EM for FIA. The proposed attacks have been quite diverse in nature, in terms of the targeted operation, method of constructing queries to the target device, as well as the mathematical approach for key recovery.

There are existing works such as References [45, 58] that have provided a good overview of existing implementations of PQC. However, a similar overview that covers recent developments in the field of SCA and FIA of PQC, and in particular, lattice-based schemes is missing. This is especially important, given the ever-growing list of attacks proposed on practical embedded implementations of lattice-based schemes. The type of attack that is applicable to a given instance of Kyber or Dilithium, depends upon a variety of factors such as the target procedure, target operation within the procedure, attack vector, operating mode of the scheme, experimental setup and many more. Thus, it is important for a designer to understand the applicability of different attacks for his/her use-case. This is especially important if a designer is tasked with choosing the right countermeasures to provide concrete protection against SCA and FIA.

There has also been significant interest in the cryptographic community towards development of SCA and FIA countermeasures for lattice-based schemes. They can be broadly classified into two categories: (1) Generic and (2) Custom. Generic countermeasures attempt to provide concrete security guarantees agnostic to the attack strategy, while custom countermeasures are those that offer protection against specific targeted attacks. With respect to SCA, there have been several

works that have proposed generic masking strategies for lattice-based schemes [10, 13, 49]. However, one can observe several shortcomings with respect to adopting generic countermeasures such as masking. First, practical attacks have been demonstrated over masked implementations of lattice-based schemes [46], and non-trivial flaws in theoretically secure masking schemes have also been exploited for key-recovery [11]. Second, masking has been shown to result in significant performance overheads for both lattice-based KEMs as well as signature schemes, especially on embedded software platforms [13, 32, 43].

In this respect, the contribution of our work is as follows:

- (1) We present the first systematic study of SCA/FIA mounted on lattice-based schemes, with main focus on two leading candidates based on variants of the LWE problem: Kyber KEM [5] and Dilithium signature scheme [24]. Through our study, we attempt to understand the underlying similarities and differences between the existing attacks while classifying them into different categories. We also discuss appropriate countermeasures for every attack discussed in this work.
- (2) While there are proposals for countermeasures for existing SCA and FIA on Kyber and Dilithium, we are not aware of a concrete implementation that incorporate multiple countermeasures into a single design. Moreover, there are existing attacks for which countermeasures are unknown or not clear. We also therefore propose and implement a few novel countermeasures for a few attacks in this work. In particular, we implement and evaluate novel countermeasures that offer simultaneous protection against several SCA- and FIA-based chosen-ciphertext attacks for Kyber KEM.
- (3) Finally, we also implement all the presented countermeasures in this work within two well-known public software libraries for PQC: (1) *pqm4* library for the ARM Cortex-M4-based microcontroller [37] and (2) *liboqs* library [68] for the Raspberry Pi 3 Model B Plus based on the ARM Cortex-A53 processor. Our performance evaluation reveals that the presented custom countermeasures incur reasonable performance overheads on both the evaluated embedded platforms. We therefore believe our work argues for usage of custom countermeasures within real-world implementations of lattice-based schemes, either in a standalone manner or as reinforcements to generic countermeasures such as masking.

Organization of the Article. In Section 2, we provide a generic description of Kyber and Dilithium. In Sections 3 and 4, we describe the known side-channel attacks and fault attacks, respectively, along with appropriate countermeasures applicable to Kyber KEM. In Sections 5 and 6, we describe the known fault-injection attacks and side-channel attacks, respectively, along with appropriate countermeasures applicable to Dilithium signature scheme. In Section 7, we demonstrate performance evaluation of all the different countermeasures against SCA and FIA for both Kyber and Dilithium. In Section 8, we provide concluding remarks for the article.

Availability of software. For scrutiny and reproducibility, we have made our implementation software available at https://github.com/PRASANNA-RAVI/SCA_FIA_Protected_Kyber_Dilithium.

2 BACKGROUND

2.1 Notations

Elements in the integer ring $\mathbb{Z}_q$ are denoted by regular font letters, viz., $a, b \in \mathbb{Z}_q$, where q is a prime. The i th bit in an element $x \in \mathbb{Z}_q$ is denoted as x_i . Vectors and matrices of integers in $\mathbb{Z}_q$, i.e., $\mathbb{Z}_q^k$ and $\mathbb{Z}_q^{k \times \ell}$ are denoted in bold upper case letters. The polynomial ring $\mathbb{Z}_q(x)/\phi(x)$ is denoted as R_q , where $\phi(x) = (x^n + 1)$ is its reduction polynomial. We denote $\mathbf{r} \in R_q^{k \times \ell}$ as a *module* of

dimension $k \times \ell$. Polynomials in R_q and vectors of polynomials in R_q^k are denoted in bold lower-case letters. Matrices of integers of polynomials, i.e., $R_q^{k \times \ell}$ are denoted in bold upper-case letters. The i th coefficient of a polynomial $\mathbf{a} \in R_q$ is denoted as $\mathbf{a}[i]$ and the i th polynomial of a given module $\mathbf{x} \in R_q^k$ as $\mathbf{x}_i$. Multiplication of two polynomials $\mathbf{a}$ and $\mathbf{b}$ in the ring R_q is denoted as $\mathbf{c} = \mathbf{a} \cdot \mathbf{b} \in R_q$ or $\mathbf{a} \times \mathbf{b} \in R_q$. Byte arrays of length n are denoted as $\mathcal{B}^n$. The i th byte in a byte array $\mathbf{x} \in \mathcal{B}^*$ is denoted as $\mathbf{x}[i]$. Pointwise/Coefficient-wise multiplication of two polynomials $(\mathbf{a}, \mathbf{b}) \in R_q$ is denoted as $\mathbf{c} = \mathbf{a} \circ \mathbf{b} \in R_q$. For a given element $\mathbf{a}$ ($\mathbb{Z}_q$ or R_q or $R_q^{k \times \ell}$), its corresponding faulty value is denoted as $\mathbf{a}^*$, and we utilize this notation throughout the article. The NTT representation of a polynomial $\mathbf{a} \in R_q$ is denoted as $\hat{\mathbf{a}} \in R_q$, and the same notation also applies to modules of higher dimension.

2.2 The Learning with Errors Problem [65]

The hardness of both Kyber and Dilithium are based on variants of the well-known **Learning With Errors (LWE)** problem. The central component of the LWE problem is the LWE instance.

Definition 2.1 (LWE Instance). For a given dimension $n \geq 1$, elements in $\mathbb{Z}_q$ with $q > 2$ and a Gaussian error distribution $\mathcal{D}_\sigma(\cdot)$, an LWE instance is defined as the ordered pair $(\mathbf{A}, T) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ where $\mathbf{A} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$ and $T = \mathbf{A} \cdot \mathbf{S} + E$ with $\mathbf{S} \leftarrow \mathcal{D}_\sigma(\mathbb{Z}_q^n)$ and $E \leftarrow \mathcal{D}_\sigma(\mathbb{Z}_q)$.

Given an LWE instance, one can define two variants of the LWE problem: (1) *Search LWE* problem - Given polynomially many LWE instances $(\mathbf{A}, T) \in (\mathbb{Z}_q^n, \mathbb{Z}_q)$, solve for $\mathbf{S} \in \mathbb{Z}_q^n$ and (2) *Decisional LWE* - Given many random instances belonging to either valid LWE instances $(\mathbf{A}, T) \in (\mathbb{Z}_q^n, \mathbb{Z}_q)$ or uniformly random instances drawn from $\mathcal{U}(\mathbb{Z}_q^n \times \mathbb{Z}_q)$, distinguish the valid LWE instances from randomly selected ones.

Cryptographic schemes built upon the standard LWE problem suffered from quadratic key sizes and computational times in the dimension n of the lattice, i.e., $O(n^2)$ [65]. Thus, most of the lattice-based schemes, especially those in the NIST standardization process, are based on *algebraically structured* variants of the standard LWE and LWR problem known as the **Ring/Module-LWE (RLWE/MLWE)** problems, respectively. The **ring variant of the LWE problem (RLWE)** [41] deals with computation over polynomials in polynomial rings $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $\mathbf{s}, \mathbf{e} \leftarrow \mathcal{D}_\sigma(R_q)$ such that the corresponding RLWE instance is defined as $(\mathbf{a}, \mathbf{t} = \mathbf{a} \times \mathbf{s} + \mathbf{e}) \in (R_q \times R_q)$. The module variant deals with computations over vectors/matrices of polynomials in $R_q^{k_1 \times k_2}$ with $(k_1, k_2) > 1$. With $\mathbf{A} \leftarrow \mathcal{U}(R_q^{k_1 \times k_2})$ and $\mathbf{s} \leftarrow \mathcal{D}_\sigma(R_q^{k_2})$ and $\mathbf{e} \leftarrow \mathcal{D}_\sigma(R_q^{k_1})$, the corresponding MLWE instance is defined as $(\mathbf{a}, \mathbf{t} = \mathbf{a} \times \mathbf{s} + \mathbf{e}) \in (R_q^{k_1 \times k_2}, R_q^{k_2})$.

2.3 Number Theoretic Transform (NTT)-based Polynomial Multiplication

Polynomial multiplication is one of the most computationally intensive operations in structured lattice-based schemes such as Kyber and Dilithium. Both Kyber and Dilithium are designed with parameters that allow the use of the well-known **Number Theoretic Transform (NTT)** for polynomial multiplication. The NTT is simply a bijective mapping for a polynomial $\mathbf{p} \in R_q$ from a *normal* domain into an alternative representation $\hat{\mathbf{p}} \in R_q$ in the *NTT domain* as follows:

$$\hat{\mathbf{p}}[j] = \sum_{i=0}^{n-1} \mathbf{p}[i] \cdot \omega^{i \cdot j}, \quad (1)$$

where $j \in [0, n-1]$ and ω is the n th root of unity in the operating ring $\mathbb{Z}_q$. The corresponding inverse operation named **Inverse NTT (INTT)** maps $\hat{\mathbf{p}}$ in the NTT domain back to $\mathbf{p}$ in the normal domain. The use of NTT requires the presence of either the n th root of unity (ω) or $2n^{\text{th}}$ root of

unity (ψ) in $\mathbb{Z}_q$ ($\psi^2 = \omega$), which can be ensured through appropriate choices for the parameters (n, q) . The powers of ω and ψ that are used within the NTT computation are commonly referred to as *twiddle constants*. NTT-based multiplication of two polynomials $\mathbf{a}$ and $\mathbf{b}$ in R_q is typically done as follows:

$$\mathbf{c} = \text{INTT}(\text{NTT}(\mathbf{a}) \circ \text{NTT}(\mathbf{b})). \quad (2)$$

The NTT over an n point sequence is performed using the well-known *butterfly* network, which operates over $\log_2(n)$ stages. Refer to the algorithmic specification document of Kyber and Dilithium for more information about the NTT used in the respective schemes [5, 24].

2.4 Kyber

2.4.1 Algorithmic Description. Kyber is a **chosen-ciphertext secure (CCA-secure)** KEM based on the Module-LWE problem that has been selected for standardization of PQC-based KEMs, owing to its strong theoretical security guarantees and implementation performance [5]. Computations are performed over modules in dimension $(k \times k)$ (i.e. $R_q^{k \times k}$). Kyber provides three security levels with Kyber512 (NIST Security Level 1), Kyber768 (Level 3), and Kyber1024 (Level 5) with $k = 2, 3$, and 4 , respectively. Kyber operates over the anti-cyclic ring R_q with a prime modulus $q = 3,329$ and degree $n = 256$, which allow the use of **Number Theoretic Transform (NTT)** for polynomial multiplication. The CCA-secure Kyber contains in its core a chosen-plaintext secure encryption scheme of Kyber (IND-CPA secure Kyber PKE), which is based on the well-known framework of the LPR encryption scheme [41].

Refer to Algorithm 1 for a simplified description of the key-generation, encryption and decryption procedures of IND-CPA secure Kyber PKE. The function Sample_U samples from a uniform distribution, Sample_B samples from a binomial distribution; Expand expands a small seed into a uniformly random matrix in $R_q^{k \times k}$. The function $\text{Compress}(u, d)$ lossily compresses $u \in \mathbb{Z}_q$ into $v \in \mathbb{Z}_{2^d}$ with $q > 2^d$, while $\text{Decompress}(v, d)$ extrapolates $v \in \mathbb{Z}_{2^d}$ into $u' \in \mathbb{Z}_q$.

Security and Correctness of IND-CPA Secure Kyber PKE

The key-generation procedure of Kyber PKE simply involves generation of an LWE instance $(\mathbf{A}, \mathbf{t}) \in (R_q^{k \times k} \times R_q^k)$, where $\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ (Line 9 in Algorithm 1). The module $\mathbf{A}$ is sampled from a uniform distribution (Line 4), while the secret $\mathbf{s}$ and errors $\mathbf{e}$ are sampled from the CBD distribution (Lines 5–6). Given that NTT is used for polynomial multiplication, the public key and secret key are directly represented in the NTT domain (Line 10). The LWE instance $(\mathbf{A}, \mathbf{t})$ is the public key, while the secret $\mathbf{s}$ forms the secret key.

The encryption procedure involves generation of two LWE instances $(\mathbf{u}, \mathbf{v}) \in (R_q^k \times R_q^k)$. The first LWE instance is generated as $\mathbf{u} = \mathbf{A}^T \cdot \mathbf{r} + \mathbf{e}_1$ (Line 18) and the second LWE instance is generated as $\mathbf{v}_p = \mathbf{t}^T \cdot \mathbf{r} + \mathbf{e}_2$ (Line 19). The message to be encrypted, i.e., $m \in \mathcal{B}^*$, is encoded into a message polynomial $\mathbf{m} \in R_q$ one bit at a time. This is done using the Encode function in the following manner (Line 20): If a message bit $m_i = 1$, then the corresponding coefficient $\mathbf{m}[i] = \lceil q/2 \rceil$, else $\mathbf{m}[i] = 0$ otherwise. Then, this message polynomial $\mathbf{m}$ is additively hidden within $\mathbf{v}_p$ as $\mathbf{v} = \mathbf{v}_p + \mathbf{m}$ (Line 20). Subsequently, the coefficients of $\mathbf{u}$ and $\mathbf{v}$ are lossily compressed to varying degrees, i.e., d_1 and d_2 bits, respectively, using the Compress function, and the compressed versions of $\mathbf{u}, \mathbf{v}$ form the ciphertext ct (Line 21).

The decryption procedure lossily extract the polynomials $\mathbf{u}'$ and $\mathbf{v}'$ from the ciphertext ct with $\Delta \mathbf{u} = (\mathbf{u}' - \mathbf{u})$ and $\Delta \mathbf{v} = (\mathbf{v}' - \mathbf{v})$ (Line 24). Subsequently, the decryption procedure computes $\mathbf{m}' = \mathbf{v}' - \mathbf{u}' \cdot \mathbf{s}$ (Lines 25–27), which is nothing but an approximation of the message polynomial

ALGORITHM 1: CPA Secure Kyber PKE (Simplified)

```

1: procedure CPA.KeyGen
2:    $seed_A \in \mathcal{B} \leftarrow \text{Sample}_U()$  ▷ Generate uniform  $Seed_A$ 
3:    $seed_B \in \mathcal{B} \leftarrow \text{Sample}_U()$  ▷ Generate uniform  $Seed_B$ 
4:    $\hat{A} = \text{NTT}(A) \in R_q^{k \times k} \leftarrow \text{Expand}(seed_A)$  ▷ Expand  $seed_A$  into  $\hat{A}$  in NTT domain
5:    $s \in R_q^k \leftarrow \text{Sample}_B(seed_B, coins_s)$  ▷ Sample secret  $s$  using  $(Seed_B, coins_s)$ 
6:    $e \in R_q^k \leftarrow \text{Sample}_B(seed_B, coins_e)$  ▷ Sample error  $e$  using  $(Seed_B, coins_e)$ 
7:    $\hat{s} \in R_q^k \leftarrow \text{NTT}(s)$  ▷ NTT( $s$ )
8:    $\hat{e} \in R_q^k \leftarrow \text{NTT}(e)$  ▷ NTT( $e$ )
9:    $\hat{t} = \hat{A} \circ \hat{s} + \hat{e}$  ▷  $t = A \cdot s + e$  in NTT domain
10:  Return  $(pk = (seed_A, \hat{t}), sk = (\hat{s}))$ 
11: end procedure

```

```

12: procedure CPA.Encrypt( $pk, m \in \{0, 1\}^{256}, seed_R \in \{0, 1\}^{256}$ )
13:    $\hat{A} \in R_q^{k \times k} \leftarrow \text{Expand}(seed_A)$ 
14:    $r \in R_q^k \leftarrow \text{Sample}_B(seed_R, coins_0)$  ▷ Sample  $r$  using  $(Seed_R, coins_0)$ 
15:    $e_1 \in R_q^k \leftarrow \text{Sample}_B(seed_R, coins_1)$  ▷ Sample  $e_1$  using  $(Seed_R, coins_1)$ 
16:    $e_2 \in R_q^k \leftarrow \text{Sample}_B(seed_R, coins_2)$  ▷ Sample  $e_2$  using  $(Seed_R, coins_2)$ 
17:    $\hat{r} \in R_q^k \leftarrow \text{NTT}(r)$  ▷ NTT( $r$ )
18:    $u \in R_q^k \leftarrow \text{INTT}(\hat{A}^T \circ \hat{r}) + e_1$  ▷  $u = A^T \cdot r + e_1$ 
19:    $v_p \in R_q \leftarrow \text{INTT}(\hat{t}^T \circ \hat{r}) + e_2$  ▷  $v_p = t^T \cdot r + e_2$ 
20:    $v = v_p + \text{Encode}(m)$ 
21:   Return  $ct = \text{Compress}(u, d_1), \text{Compress}(v, d_2)$ 
22: end procedure

```

```

23: procedure CPA.Decrypt( $sk, ct$ )
24:    $u' \in R_q^k = \text{Decompress}(u, d_1); v' \in R_q^k = \text{Decompress}(v, d_2)$ 
25:    $\hat{u}' = \text{NTT}(u')$ 
26:    $\hat{g}' = \hat{u}' \circ \hat{s}$ 
27:    $m' \in R_q = v' - \text{INTT}(\hat{g}')$  ▷  $m' = v' - u' \cdot s$ 
28:    $m' \in \mathcal{B}^* = \text{Decode}(m')$ 
29:   Return  $m'$ 
30: end procedure

```

m , i.e., m' , which is given as follows:

$$\begin{aligned}
m' &= v' - s^T \cdot u' \\
&= (v + \Delta v) - s^T \cdot (u + \Delta u) \\
&= (t^T \cdot r + e_2 + \text{Encode}(m) + \Delta v) - s^T \cdot (A^T \cdot r + e_1 + \Delta u) \\
&= \text{Encode}(m) + (e^T \cdot r + \Delta v + e_2 - s^T \cdot e_1 - s^T \cdot \Delta u) \\
&= \text{Encode}(m) + d,
\end{aligned} \tag{3}$$

where $d = (e^T \cdot r + \Delta v + e_2 - s^T \cdot e_1 - s^T \cdot \Delta u)$ is the noise component in m' , which is also linearly dependent on the secret and error (s, e) of the public-private key pair. The approximate

```

1 uint16_t t = (((m->coeffs[8*i+j] << 1) + KYBER_Q/2) / KYBER_Q) & 1;
2 m[i] |= t << j;

```

Fig. 1. Message decoding routine in Kyber KEM, which converts the message polynomial $\mathbf{m} \in R_q$ into a 32-byte message array m , where i denotes the byte location and j denotes the bit location within a given byte.

message polynomial $\mathbf{m}'$ is decoded into the message $m' \in \mathcal{B}^*$ one bit at a time in the following manner: If a given message coefficient $\mathbf{m}[i]$ is in the range $[q/4, 3q/4]$, then $m_i = 1$, else $m_i = 0$ otherwise (Line 28). This is computed using a specialized decoding routine, which is sketched in the code snippet shown in Figure 1. It takes as input the message polynomial $\mathbf{m}$ and decodes the coefficients one at a time into corresponding bits in the 32-byte message array m .

As long as the absolute value of all the coefficients of the noise $\mathbf{d}$ are less than $q/4$, i.e., $\ell_\infty(\mathbf{d}) < q/4$, the message polynomial $\mathbf{m}'$ is decoded to the correct message m , i.e., $m' = m$. The parameters of the scheme are chosen to attain a negligible decryption failure probability. For recommended parameters of Kyber, the decryption failure probability is $\approx 2^{-164}$. While we have only presented a simplified description of Kyber PKE, a more detailed description can be found in Reference [5].

2.4.2 Security against Chosen-ciphertext Attacks. The aforementioned PKE is only secure against chosen-plaintext attacks (IND-CPA security) and thus is not secure against chosen-ciphertext attacks. These attacks typically work by querying the decryption procedure with malicious and invalid ciphertexts and obtaining information about the corresponding decrypted message m' . This information about m' for malicious and invalid ciphertexts can be used to recover the complete secret key.

The CPA secure Kyber PKE is converted into a CCA secure KEM using the well-known Fujisaki-Okamoto transformation [26]. It utilizes a pair of hash functions $\mathcal{H}$ and $\mathcal{G}$ and a key-derivation function KDF and forms a wrapper around the encryption and decryption procedures, resulting in encapsulation and decapsulation procedures of a CCA secure KEM (refer to Algorithm 2).

In theory, the FO transform helps protect the decapsulation procedure of KEMs against chosen-ciphertext attacks in the following manner: The message m' obtained after decryption of the received ciphertext ct (Line 17) is hashed with the public key to generate a pre-shared secret $\bar{K}'$ and a seed r' (Line 18). The message m' along with the seed r' is then fed into a re-encryption procedure to recompute the ciphertext as ct_R (Line 20). A subsequent comparison of ct_R with the received ciphertext ct helps evaluate the validity of ct (Line 21). For a valid ciphertext, $ct = ct_R$ with a very high probability, and as a result, a valid shared secret K dependent upon the pre-shared secret $\bar{K}'$ and the received ciphertext ct_R is generated (Line 24). However, for an invalid ciphertext, comparison fails with an overwhelming probability, resulting in generation of a pseudo-random secret K , using a pseudo-random value z and the received ciphertext ct (Line 22, 24). Thus, for invalid ciphertexts, an attacker cannot obtain any information about the decrypted message m' , which provides concrete protection against chosen-ciphertext attacks.

2.5 Dilithium

Dilithium is a lattice-based signature scheme secure whose security is based on the **Module LWE (M-LWE)** and **Module SIS (M-SIS)** problem [24]. Dilithium operates over the module $R_q^{k \times \ell}$ with $(k, \ell) > 1$ where $R_q = \mathbb{Z}[x]/(x^n + 1)$, $n = 256$ and $q = 2^{23} - 2^{17} - 1$. This choice of parameters allows the use of NTT for polynomial multiplication in R_q . Dilithium also comes in three security levels: Dilithium2 with $(k, \ell) = (4, 4)$ at NIST Level 2, Dilithium3 with $(k, \ell) = (6, 5)$ at NIST Level 3, and Dilithium5 with $(k, \ell) = (8, 7)$ at NIST Level 5. There are two variants of Dilithium:

ALGORITHM 2: FO transform of a CPA-secure Kyber PKE into a CCA-secure Kyber KEM

```

1: procedure CCA.KeyGen
2:    $z \leftarrow \{0, 1\}^{256}$ 
3:    $(pk, sk') \leftarrow \text{CPA.KeyGen}()$ 
4:    $sk = (sk' || \mathcal{H}(pk) || z)$ 
5:   Return  $(pk, sk)$ 
6: end procedure

```

```

7: procedure CCA.Encaps( $pk$ )
8:    $m \leftarrow \{0, 1\}^{256}$ 
9:    $m = \mathcal{H}(m)$ 
10:   $(\tilde{K}, r) = \mathcal{G}(m || \mathcal{H}(pk))$  ▷ Generation of pre-key  $\tilde{K}$ 
11:   $ct = \text{CPA.Encrypt}(pk, m, r)$  ▷ Encryption of message  $m$  using public key  $pk$ 
12:   $K = \text{KDF}(\tilde{K} || \mathcal{H}(ct))$  ▷ Generation of session key
13:  Return  $(ct, K)$ 
14: end procedure

```

```

15: procedure CCA.Decaps( $sk, ct$ )
16:   $(pk, \mathcal{H}(pk), z) \leftarrow \text{UnpackSK}(sk)$ 
17:   $m' = \text{CPA.Decrypt}(sk, ct)$  ▷ Decryption of ciphertext into message
18:   $(\tilde{K}', r') = \mathcal{G}(m', \mathcal{H}(pk))$  ▷ Generation of pre-key  $\tilde{K}'$ 
19:   $T = \tilde{K}'$ 
20:   $ct_R = \text{CPA.Encrypt}(pk, m', r')$  ▷ Re-Encryption of decrypted message
21:  if  $(\text{CompareCT}(ct_R, ct) == 0)$  then ▷ Ciphertext Comparison
22:     $T = z$  ▷ Ciphertext Comparison Failure
23:  end if
24:  Return  $K = \text{KDF}(T || \mathcal{H}(ct))$  ▷ Generation of session key
25: end procedure

```

(1) Deterministic (2) Probabilistic/Randomized, which only subtly differ in the way randomness is used in the signing procedure. The signing procedure of the deterministic Dilithium does not utilize external randomness and can generate only a single signature for a given message. The randomized variant, however, utilizes external randomness and thus generates a different signature for a given message in each execution.

2.5.1 Algorithmic Description. Refer to Algorithms 3–4 for a simplified description of the key generation, signing, and verification procedures of Dilithium. The functions Sample_U , Sample_B , and Expand perform the same functions as in Kyber, albeit with different parameters. Dilithium also uses a number of rounding functions such as Power2Round , HighBits , LowBits , MakeHint , and UseHint , whose details can be found in Reference [24]. The key generation procedure simply involves generation of an LWE instance $\mathbf{t}$ (Line 6 in Algorithm 3). Subsequently, the LWE instance is split into higher and lower order bits $\mathbf{t}_1$ and $\mathbf{t}_0$, respectively (Line 7), where $\mathbf{t}_1$ forms part of the public key, while $\mathbf{t}_0$ becomes part of the secret key.

The signing procedure of Dilithium is based on the “Fiat-Shamir with Aborts” framework, where the signature is repeatedly generated and rejected until the signature and its associated intermediate variables satisfy a given set of conditions[40]. The message m is first hashed with a public value tr to generate μ (Line 13). The abort loop (Lines 21–39) starts by generating an ephemeral nonce $y \in R_q^\ell$ using a seed ρ . For the deterministic variant, the seed ρ is obtained by hashing

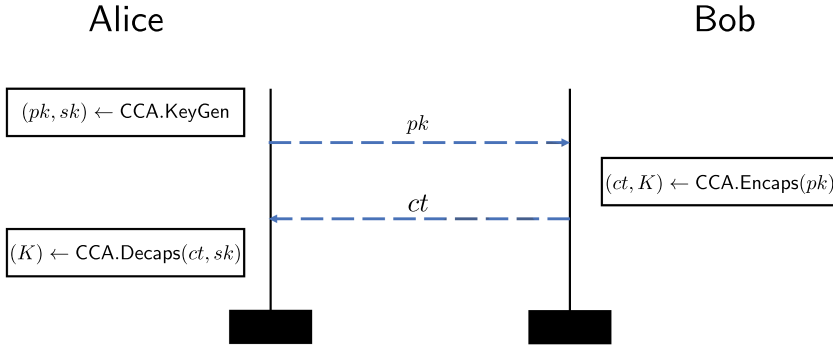


Fig. 2. Key-exchange protocol using IND-CCA secure Kyber KEM.

μ with a secret nonce K (Line 17), while the probabilistic variant randomly samples the seed ρ from a uniform distribution (Line 19). This is the only differentiator between the two variants. The nonce y along with the public key component A is then used to calculate a sparse challenge polynomial $c \in R_q$ (Line 25), whose 60 coefficients are either ± 1 , while the other 196 coefficients are 0. Subsequently, the challenge c , nonce y , and secret s_1 are used to compute the primary signature component z (Line 27). Then, a hint vector h is generated and output as part of the signature σ (Line 33). The abort loop contains several conditional checks (Lines 29, 34), which should be simultaneously satisfied to terminate the abort loop and generate the signature $\sigma = (z, h, c)$.

The verification procedure utilizes the signature σ and the public key pk to recompute the challenge polynomial $\bar{c}$ (Line 5 in Algorithm 4), which is then compared with the received challenge c , along with other checks (Line 6). If all the checks are satisfied, then the verification is successful, else it is a failure. While we have only presented a simplified description of the Dilithium signature scheme, we refer the reader to Reference [24] for a detailed description of the same.

3 SIDE-CHANNEL ATTACKS ON KYBER KEM

3.1 Nomenclature for Attack Classification

Kyber KEM has been subjected to a variety of side-channel attacks, and the type of attack that can be mounted in a given setting depends upon several factors such as target procedure, target operation, attack technique, operating mode of Kyber, and so on. Understanding the applicability of different attacks requires one to understand the application of Kyber KEM when used for key-exchange, i.e., within a key exchange protocol.

3.1.1 Application of Kyber KEM for Key-exchange. Refer to Figure 2 for a key-exchange protocol that can be built using IND-CCA secure Kyber KEM. The protocol is executed between two parties, Alice and Bob. Alice starts by running the key-generation procedure (KeyGen) to generate her public-private key pair (pk, sk) and subsequently sends the public key pk to Bob. Bob then runs the encapsulation procedure (Encaps) procedure using the public key pk to generate the ciphertext ct and the corresponding shared session key K . Bob shares the ciphertext ct with Alice, who then uses her secret key sk to decapsulate the ciphertext (Decaps) to generate the same shared session key K .

This key-exchange protocol can operate in two settings, depending upon the longevity of the key pair (pk, sk) used by Alice.

ALGORITHM 3: Dilithium Signature scheme (Simplified)

```

1: procedure KEYGEN
2:    $(seed_A, seed_S, K) \in \mathcal{B} \leftarrow \text{Sample}_U();$ 
3:    $s_1, s_2 \in (R_q^\ell \times R_q^k) \leftarrow \text{Sample}_B(seed_S)$  ▷ Generate the secrets  $s_1$  and  $s_2$ 
4:    $A \in R_q^{k \times \ell} \leftarrow \text{Expand}(seed_A)$ 
5:    $\hat{s}_1 = \text{NTT}(s_1)$  ▷ Compute NTT of  $s_1$ 
6:    $t = \text{INTT}(A \circ \hat{s}_1) + \hat{s}_2$  ▷ Generate LWE instance  $t$ 
7:    $(t_1, t_0) \leftarrow \text{Power2Round}(t)$  ▷ Split  $t$  as  $t_1 \cdot 2^d + t_0$ 
8:    $tr \in \mathcal{B} \leftarrow \mathcal{H}(seed_A \| t_1)$ 
9:    $pk = (seed_A, t_1), sk = (seed_A, K, tr, s_1, s_2, t_0)$ 
10: end procedure

```

```

11: procedure SIGN( $sk, M$ )
12:    $\hat{A} \in R_q^{k \times \ell} \leftarrow \text{Expand}(seed_A)$ 
13:    $\mu \in \{0, 1\}^{512} \leftarrow \mathcal{H}(tr \| M)$  ▷ Hash  $m$  with public value  $tr$ 
14:    $\kappa \leftarrow 0; (z, h) \leftarrow \perp$ 
15:    $\hat{s}_1 = \text{NTT}(s_1), \hat{s}_2 = \text{NTT}(s_2), \hat{t}_0 = \text{NTT}(t_0)$ 
16:   if Deterministic then
17:      $\rho \in R_q^\ell \leftarrow \mathcal{H}(K \| \mu)$  ▷ Generate seed  $\rho$  using message and secret seed  $K$ 
18:   else
19:      $\rho \in R_q^\ell \leftarrow \text{Sample}_U()$  ▷ Generate uniform seed  $\rho$ 
20:   end if
21:   while  $(z, h) = \perp$  do ▷ Start of Abort Loop
22:      $y \leftarrow \text{Sample}_Y(\rho \| \kappa)$ 
23:      $\hat{y} = \text{NTT}(y)$  ▷  $\text{NTT}(y)$ 
24:      $w \leftarrow \text{INTT}(\hat{A} \circ \hat{y}); w_1 \leftarrow \text{HighBits}(w)$  ▷  $w_1 = \text{HighBits}(A \cdot y)$ 
25:      $c \in R_q \leftarrow \mathcal{H}(\mu \| w_1)$  ▷ Generate Sparse Challenge  $c$ 
26:      $\hat{c} = \text{NTT}(c)$  ▷  $\text{NTT}(c)$ 
27:      $z = \text{INTT}(\hat{c} \circ \hat{s}_1) + y$  ▷  $z = s_1 \cdot c + y$ 
28:      $r_0 = \text{LowBits}(w - c \cdot s_2)$ 
29:     if  $\|z\|_\infty \geq \gamma_1 - \beta$  or  $\|r_0\|_\infty \geq \gamma_2 - \beta$  then ▷ Conditional Checks
30:        $(z, h) = \perp$ 
31:        $\kappa = \kappa + 1$ 
32:     else
33:        $h = \text{MakeHint}(-c \cdot t_0, w - cs_2 + c \cdot t_0, 2\gamma_2)$ 
34:       if  $\|c \cdot t_0\|_\infty \geq \gamma_2$  or  $\#1\text{'s in } h > \omega$  then ▷ Conditional Checks
35:          $(z, h) = \perp$ 
36:          $\kappa = \kappa + 1$ 
37:       end if
38:     end if
39:   end while
40:    $\sigma = (z, h, c)$ 
41: end procedure

```

- (1) *Static key setting:* Alice can choose to reuse (pk, sk) for multiple key-exchanges, and this is referred to as a static-key setting with occasional key refreshment (once every X

ALGORITHM 4: Dilithium Signature scheme (Simplified)

```

1: procedure VERIFY( $pk, M, \sigma = (z, h, c)$ )
2:    $\mu \in \{0, 1\}^{512} \leftarrow \mathcal{H}(tr||M)$ 
3:    $\hat{c} = \text{NTT}(c)$ 
4:    $w'_1 := \text{UseHint}(h, A \cdot z - \text{INTT}(\hat{c} \circ \hat{t}_1 \cdot 2^d, 2\gamma_2))$  ▷ Generating  $w'_1$ 
5:    $\bar{c} = \mathcal{H}(\mu, w'_1)$  ▷ Recomputing Challenge polynomial
6:   if ( $\bar{c} == c$ ) and (norm of  $z$  and  $h$  are valid) then
7:     Return Pass
8:   else
9:     Return Fail
10:  end if
11: end procedure

```

key-exchanges). It is also possible that Alice also uses a single key pair for all key-exchanges without any key refreshment. For simplicity, we refer to these scenarios together as the static key setting.

- (2) *Ephemeral key setting*: However, Alice can also choose to use fresh key pairs (pk, sk) for every new key-exchange, which we refer to as the ephemeral key setting. In the ephemeral key setting, IND-CCA security is not required, and thus the key-exchange can be carried out only using IND-CPA secure Kyber PKE. Thus, Bob and Alice can utilize the CPA.Encrypt and CPA.Decrypt procedures, respectively, instead of the CCA.Encaps and CCA.Decaps to run the key-exchange protocol in the ephemeral key setting. However, key-exchange in this setting can also be carried out using IND-CCA secure Kyber KEM, but this is generally considered to be an overkill.

The type of attacks on Kyber KEM, primarily depend upon the interplay between the following two factors:

- (1) Kyber's operating mode - ephemeral/static key setting
- (2) Attacker's access to **Device Under Test (DUT)** - Alice/Bob

3.1.2 Attacking Kyber in Ephemeral Key Setting. In this setting, the secret key sk and sensitive message m are refreshed for every new key exchange. Recovery of m leads to recovery of the session key K . Recovery of sk also leads to recovery of the session key K . This allows the attacker to decrypt future dated secure communication between Alice and Bob, encrypted using the session key K . Thus, in the ephemeral key setting, recovering the message m (message recovery) is equivalent to recovering the secret key sk (key recovery).

- (1) *Attacking Alice*: If an attacker has access to Alice's device, then he/she can target the key-generation or decapsulation procedure. Leakage from the key-generation procedure can be exploited to recover the secret key sk . One of the main challenges of targeting the key-generation procedure is that it is probabilistic and generates a new key pair for every execution. Thus, the attacker only has access to a single execution/trace of the key-generation procedure to recover the entire secret key sk . Thus, multi-trace side-channel attacks or fault attacks that require multiple faulty outputs naturally do not apply to the key-generation procedure. However, in the ephemeral key setting, key-generation procedure is executed for every new instance of the key-exchange protocol, thus an attacker has the opportunity to attack key-generation every time a key-exchange is initiated by Alice.

An attacker can also exploit leakage from the decapsulation procedure for key recovery (sk) as well as message recovery m . Similar to targeting the key-generation procedure, the attacker only has a single trace/execution of the decapsulation procedure to recover the entire secret key sk . Thus, only single trace/execution attacks apply, while multi-trace attacks do not apply.

- (2) *Targeting Bob*: If an attacker has access to Bob's device in the ephemeral key setting, then he/she can target the encapsulation procedure for message recovery, i.e., m . The encapsulation procedure is also probabilistic, and thus the attacker only has access to a single execution to recover the entire message m .

3.1.3 Attacking Kyber in Static Key Setting. In this setting, recovering the secret key sk is much more attractive for an attacker, since recovering sk results in trivial recovery of the message m for all the key-exchanges carried out by Alice using sk . Thus, recovery of a single long-term secret key sk leads to recovery of all the session keys K derived using sk .

- (1) *Targeting Alice*: An attacker can target Alice's key-generation and decapsulation procedure. Unlike the ephemeral key setting where the key-generation procedure is executed for every key-exchange, key-generation is only executed once every X times, where X is the key refresh rate chosen by the designers based on the application. Thus, the attacker has lesser opportunity to attack key-generation in a static setting with occasional key refreshes, compared to the ephemeral key setting. Moreover, only single trace/execution attacks are applicable to the key-generation procedure.

However, in a static key setting, the decapsulation procedure serves as a better target for the attacker, since it manipulates the same secret key sk for X key-exchanges. Thus, an attacker has access to X number of executions of the decapsulation procedure to recover the secret key sk , compared to only a single trace/execution in the ephemeral key setting. Thus, multi-trace attacks apply when targeting the decapsulation procedure in a static key setting.

- (2) *Targeting Bob*: If an attacker has access to Bob's device, then he/she can target the encapsulation procedure for message recovery. Similar to the ephemeral key setting, only single trace attacks apply to the encapsulation procedure.

From the aforementioned discussion, we can infer the following:

- (1) Key-generation procedure and encapsulation procedure can only be targeted by single trace attacks, irrespective of operating in the static key setting or ephemeral key setting.
- (2) In ephemeral key setting, decapsulation procedure can be targeted only by single trace attacks. However, in the static key setting, multi trace attacks apply to the decapsulation procedure.

3.1.4 Attack Scenarios and Characteristics. We therefore discuss side-channel attacks on Kyber KEM based on four scenarios:

- (1) Targeting Key-generation Procedure (Single Trace)
- (2) Targeting Encapsulation Procedure (Single Trace)
- (3) Targeting Decapsulation Procedure in Ephemeral Key Setting (Single Trace)
- (4) Targeting Decapsulation Procedure in Static Key Setting (Multi Trace)

For every attack discussed in this work, we also describe its characteristics based on the following parameters:

- (1) *Attacker's ability to communicate with DUT* (DUT_IO_Access): In this respect, we identify two categories:

- (a) *Observe_DUT_IO*: We assume an attacker who can only passively observe the DUT's communication channel (I/O) but cannot actively communicate with the DUT. In this scenario, the attackers can only observe/affect the DUT's behavior for valid key-exchanges with other parties. The attacker cannot trigger operation of the DUT.
 - (b) *Communicate_DUT_IO*: We assume an attacker who can passively observe the DUT's communication channel while also being able to actively communicate with the DUT. For instance, when targeting Alice, an attacker can attempt to establish key-exchange and submit ciphertext queries to observe the behavior of Alice. Similarly, when targeting Bob, an attacker can attempt to perform a valid key-exchange with Bob to observe the behavior of his DUT. This ability to communicate with the DUT provides the attacker the opportunity to profile the side-channel behavior of the DUT when processing attacker's chosen inputs. As we show later in Section 3.6, this serves as an advantage for an attacker, leading to more variety of attacks.
- (2) *Profiling and Access to clone device (Profile_Requirement)*: In this respect, we can categorize attacks into three categories:
- (a) *Profiled_With_Clone*: This category includes profiled attacks, which work with side-channel templates built using leakage from a clone device of the DUT. Thus, the attacker requires access to a clone device, which he/she can fully control, including the secret key. The attacker can construct elaborate side-channel templates, using leakage from the clone device, for every operation done on the DUT.
 - (b) *Profiled_Without_Clone*: This category includes profiled attacks, which can work with templates constructed using leakage, directly from the DUT. Thus, these attacks do not require access to a clone device.
 - (c) *Non_Profiled*: This category includes attacks that do not utilize any side-channel templates, thus these attacks also do not require access to a clone device.
- (3) *Number of traces (No_Traces)*: This characteristic denotes the number of traces from the DUT, required to perform message recovery/key recovery. We do not take into account the number of traces, required to construct side-channel templates. We only consider the number of executions of the DUT to carry out the attack. The exact number of traces for key/message recovery depends upon the experimental setup, and therefore, we only provide approximate numbers for the same in this article, while the main emphasis is on the scale of the number of traces, rather than the exact number.
- (4) ***Signal to Noise Ratio (SNR)***: This characteristic indicates the robustness of side-channel attack to measurement noise in the acquired traces. We identify two categories: *Low_SNR* and *High_SNR*. We clarify that the SNR comparison is only qualitative and that attacks that work over multiple traces typically require lower SNR, compared to attacks that work with single traces.

We therefore define the characteristic of each side-channel attack on Kyber using the following tuple: (*DUT_IO_Access*, *Profile_Requirement*, *No_Traces*, *SNR*). For example, a tuple (*Observe_DUT_IO*, *Profiled_With_Clone*, 1, *High_SNR*) indicates a side-channel attack with the following characteristics: one that does not require to communicate with the DUT, requires a clone device to construct templates, only requires a single trace from the DUT, and requires a reasonably high SNR in the collected measurements. We believe this representation captures the high-level features about the attack and allows us to categorize existing attacks into different categories.

To explain the different attacks, we utilize the algorithm of IND-CPA secure Kyber PKE in Algorithm 1 and algorithm of IND-CCA secure Kyber KEM in Algorithm 2. In this article, we consider power/EM side-channel to be equivalent, as attacks exploiting the power (respectively, EM)

side-channel can be easily adapted to exploit the EM (power) side-channel, albeit with appropriate difference in the success rate and effort to carry out the attack.

3.2 SCA on Key Generation

We now discuss single trace attacks applicable to the key-generation procedure of Kyber KEM.

3.2.1 Soft-Analytical Side-Channel Analysis (SASCA). In a single power/EM trace, the attacker has to extract as much information as possible from a single trace to recover the target variable. In this respect, Soft-Analytical Side-Channel Analysis (SASCA) acts as a very potent tool to perform single trace attacks. They are profiled attacks that work by constructing templates using leakage from multiple sequential operations, directly processing the secret variable. Subsequently, to carry out the attack, the attacker obtains a single trace, which is then matched with all the templates, and the template matching information is combined to recover the secret key. The ability of SASCA for key recovery was initially studied for symmetric key cryptographic schemes such as AES [72], however, its applicability to lattice-based schemes has also been studied by several works [38, 51, 53]. We can classify existing attacks based on SASCA into two categories, based on the target operation.

Targeting NTT: Primas et al. [53] showed that a single power/EM trace from the NTT operation operating over a secret variable x can be used to recover x , i.e., input to the NTT. A close observation of the algorithm of IND-CPA Kyber PKE (Algorithm 1) reveals that NTT is computed over several sensitive intermediate variables. In particular, within the key-generation procedure, NTT is computed over the secret key $s \in R_q^k$ (Line 7 of KeyGen). Thus, leakage from this NTT operation can be exploited by SASCA to recover its input, i.e., s .

The attack works in two phases: (1) Profiling Phase and (2) Key Recovery Phase.

- (1) *Profiling Phase:* Side-channel templates are constructed using leakage from the clone device (Profiled_With_Clone) for several intermediate computations within the NTT. Some of these operations include storing/loading of input and output of butterfly operation, modular addition, modular subtraction, and modular multiplication.
- (2) *Key Recovery Phase:* Once templates are constructed, the attacker obtains a single trace corresponding to leakage from the target NTT operation. The trace is segmented based on the targeted internal operations, which are then matched with the appropriate templates. Subsequently, results from the template matching are modelled into a factor graph based on the NTT implementation. The factor graph is fed into the Belief Propagation algorithm [50], which combines leakage from all the intermediate variables to recover the secret key s .

The first SASCA-based single trace attack on lattice-based schemes was proposed by Primas et al. [53] on a generic Ring-LWE-based PKE scheme, implemented on the ARM Cortex-M4 microcontroller. The proposed attack required over a million templates for successful key recovery. However, the subsequent work of Pessl and Primas [51] proposed optimizations to reduce the number of templates to just a few hundred (≈ 200), especially when the coefficients of the input to the target NTT belong to a small range. This is precisely the case with the NTT over the secret $s \in R_q^k$ in the KeyGen procedure (Line 7 in Algorithm 1). We refer to these attacks targeting NTT using SASCA by the label SASCA_NTT. Their Characteristics (plural) can be defined by the following tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Very recently, Li et al. [38] proposed single trace attacks on the reference implementation of the Toom-Cook polynomial multiplier used in Saber KEM. The aforementioned attacks clearly demonstrate the ability of SASCA-style attacks to break different algorithms for polynomial multiplication when used in lattice-based schemes.

Countermeasure: We refer to the work of Ravi et al. [61], who proposed generic shuffling and masking countermeasures with varying granularity to protect the NTT against single trace attacks. They proposed a range of shuffling countermeasures that provide a well-defined trade-off between the shuffling entropy (security) and performance. They also proposed masked NTTs, which randomize the twiddle factors used in the NTT operation. This has the effect of randomizing computations within the NTT, which deters the success rate of SASCA-type attacks. For more details, we refer to Reference [61] for the proposed masking and shuffling countermeasure for the NTT. Recently, the security offered by the shuffled NTT countermeasures was studied in a detailed manner by Hermelink et al. [34]. We refer to these countermeasures for the NTT together using the label `Shuffled_Masked_NTT`.

Targeting KECCAK: KECCAK is used as a building block in several lattice-based schemes including Kyber KEM. In particular, it is used as a **pseudo-random function (PRF)** and a **pseudo-random number generator (PRNG)** across all the three procedures of Kyber KEM (KeyGen, Encaps, and Decaps). In the KeyGen procedure, KECCAK is used as a PRNG to expand a small secret seed $seed_B$ into a string of pseudo-random bits (Line 5 in Algorithm 1), which is subsequently used to sample the secret s . Given the sequential nature of the KECCAK operation, it serves as an ideal target for SASCA.

In this respect, Kannwischer et al. [36] demonstrated single trace SASCA on KECCAK instances, which can be used to recover their inputs. Thus, targeting the KECCAK operation over the secret $seed_B$ (Line 5 in Algorithm 1) can be used to recover $seed_B$, whose knowledge can be used to recover the secret s . Though the attack was only demonstrated over simulated traces, the attack in principle is applicable to software implementations particularly on embedded microcontrollers such as the ARM Cortex-M4. We henceforth refer to the attack using the label `SASCA_KECCAK`. The templates required to perform the attack can only be built using leakage from a clone device. The attack characteristic can be defined by the tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR). KECCAK is extensively utilized as a PRF and PRNG in several lattice-based KEMs and also extensively within hash-based signatures, thus the `SASCA_KECCAK` attack is also applicable to other PQC schemes, as discussed in Reference [36].

Countermeasure: Similar to the NTT operation, KECCAK can be protected from `SASCA_KECCAK`-style attacks through shuffling. However, we are not aware of prior work that investigates the cost and effectiveness of partial or full shuffling of KECCAK instances in lattice-based schemes. We refer to the shuffling countermeasure for KECCAK as `Shuffled_KECCAK` throughout this article.

Though SASCA-based attacks targeting the NTT and KECCAK instances can work with single traces, they suffer from a few downsides:

- (1) *Requirement of Elaborate Profiling:* Several hundred precisely built templates for low-level arithmetic operations are required for a successful attack. This requires the attacker to have detailed information about the target and its internal operations.
- (2) *Requirement of High Signal to Noise Ratio (SNR):* The attack typically requires a relatively high SNR for full key recovery, which is typical of single trace attacks. Thus, incorporation of low-cost countermeasures such as jitter could already be sufficient to significantly deter the success rate of the attack.
- (3) *Applicability of Attack to Noisy Devices:* The aforementioned attacks have only been demonstrated on embedded microcontrollers such as the ARM Cortex-M4 with high SNR. But their applicability to more advanced processors with inherently low SNR is not clear. Moreover, hardware implementations with inherent parallelism introduce significant algorithmic noise, which can also significantly deter attack success rate.

Refer to Table 1 for a tabulation of all side-channel attacks on the key-generation procedure of Kyber KEM.

3.3 SCA on Encapsulation

Similar to the key-generation procedure, the encapsulation procedure is also probabilistic and is therefore only susceptible to single trace message recovery attacks in both the ephemeral key setting as well as static key setting. We now discuss single trace attacks applicable to the encapsulation procedure of Kyber KEM.

3.3.1 SASCA. The encapsulation procedure is also susceptible to SASCA-based attacks.

Targeting NTT (SASCA_NTT): Leakage from the NTT over the ephemeral secret $\mathbf{r} \in R_q^k$ (Line 17 of Encrypt in Algorithm 1) can be exploited to recover $\mathbf{r}$ in a single trace. Recovery of $\mathbf{r}$ leads to straightforward recovery of the message m for a valid ciphertext $ct = (\mathbf{u}, \mathbf{v})$ in the following manner:

$$m = \text{Compress}(\mathbf{v} - \text{INTT}(\hat{\mathbf{t}} \circ \hat{\mathbf{r}}), 1).$$

The attack can only be carried out using templates built from the clone device, since the attacker does not have knowledge of the internal variables of the target computation, i.e., $\text{NTT}(\mathbf{r})$. Thus, the attack characteristic can be defined by the following tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Targeting KECCAK (SASCA_KECCAK): KECCAK is used as a PRF as well as a PRNG within the encapsulation procedure. In the Encaps procedure, it is used as a PRF (denoted as $\mathcal{G}$) to generate the pre-key $\tilde{K}'$ and seed r using the sensitive message m and hash of the public key pk (Line 10 in Alg : CCA_transform). Thus, exploiting leakage from this KECCAK instance leads to recovery of the message m in a single trace.

In the Encrypt procedure, the KECCAK operation is used as a PRNG to expand a small secret seed, i.e., $seed_R$ (32 bytes) into a string of pseudorandom bits, which are further used to sample the ephemeral secrets $\mathbf{r}$, $\mathbf{e}_1$, and $\mathbf{e}_2$ (Lines 14–16 in Algorithm 1). Exploitation of leakage from this KECCAK instance leads to recovery of $seed_R$ in a single trace, whose knowledge can be used to recover $\mathbf{r}$ and therefore the message m . Simliar to SASCA on the NTT, the attack can be defined using the tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Countermeasure: Shuffling and masking the NTT (Shuffled_Masked_NTT) as well as shuffling the KECCAK operation (Shuffled_KECCAK) provides concrete protection against attacks relying on SASCA.

3.3.2 Targeting Message Encoding. KEMs based on the LWE/LWR-based problem such as Kyber inherently involve bitwise manipulation of the message m . During encryption, the message $m \in \mathcal{B}^{32}$ is encoded one bit at a time into a polynomial $\mathbf{m} \in R_q$ within the Encode procedure (Line 20 of Encrypt in Algorithm 1). This behavior has been exploited by several power/EM side-channel attacks for message recovery [3, 67, 76].

The first such attack was demonstrated by Amiet et al. [3] targeting the message encoding operation in NewHope KEM, a Ring-LWE-based KEM. The encoded message polynomial has only two possible coefficients, i.e., $\mathbf{m}[i] = \lceil q/2 \rceil$ for $m_i = 1$ and $\mathbf{m}[i] = 0$ for $m_i = 0$. For a non-zero modulus q , the difference in Hamming Weight of $\mathbf{m}[i]$ when $m_i = 0/1$, i.e., $\mathbf{m}[i] = 0$ or $\mathbf{m}[i] = q/2$, can be easily distinguished through the power/EM side-channel. Thus, leakage from manipulation of these encoded coefficients (Line 20) can be used to recover the message one bit at a time from a single trace.

The attack works in two phases: (1) Profiling Phase and (2) Key Recovery Phase.

- (1) *Profiling Phase*: The attacker builds templates for all message bits $m_i = 0$ and $m_i = 1$ for $i \in [0, 256]$. The templates can be constructed in two ways. If the attacker can communicate with the DUT (Communicate_DUT_IO), then he/she can perform several valid key-exchanges with the DUT to build side-channel templates for all message bits. Thus, the templates can be built directly on the DUT (Profiled_Without_Clone). However, if the attacker cannot communicate with the DUT (Observe_DUT_IO), then templates have to be built on a clone device to carry out the attack (Profiled_With_Clone).
- (2) *Key Recovery Phase*: The attacker obtains a single attack trace and divides it into smaller segments, corresponding to the individual message bits m_i for $i \in [0, 256]$. These segments are matched with the corresponding templates $m_i = 0$ and $m_i = 1$ to recover the entire message in a single trace.

While the attack was originally demonstrated on NewHope KEM, subsequent works [67, 76] have generalized the same attack to multiple lattice-based KEMs including Kyber KEM. We refer to these attacks using the label Message_Encode.

Since it is a single trace attack, masking does not serve as a concrete countermeasure against the attack. In fact, attacks have been demonstrated exploiting similar bitwise manipulation of the message, on implementations protected with first-order and higher-order masking countermeasures [46, 48]. These attacks show that an attacker can exploit leakage from all the individual shares of the message bits for single trace message recovery. The attack in effect does not really break the security guarantees of the masked implementation but is merely a second-order attack on a first-order masked implementation. However, this attack is of interest, because it is expected that the number of traces required for the attack increases exponentially with the masking order. However, Ngo et al. [46] showed that single trace message recovery is also possible on a first-order masked implementation, similar to the unprotected implementation. Similarly, the same attack was extended to higher orders by the same authors [48], thereby clearly demonstrating that masking does not deter message recovery when the message is manipulated in a bitwise fashion. Though these attacks exploited the message decoding operation within the decoding procedure of Saber KEM (equivalent to Line 28 of Decrypt in Algorithm 1), the same attacks also apply to the encoding procedure.

We refer to these attacks applicable to the masked message encoding procedure as Masked_Message_Encode. All the aforementioned attacks have been demonstrated on the ARM Cortex-M4 microcontroller, where the encoding operation is done in a sequential manner, one bit at a time. However, the applicability of these single trace attacks to parallelized implementations is not clear. Moreover, leakage from manipulation of single message bits only span for a single clock cycle. Thus, exploitation of such fine-grained leakage requires traces with sufficiently high SNR for message recovery.

The characteristic of both the Message_Encode and Masked_Message_Encode attacks on the encapsulation procedure can be described using these two tuples: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR) and (Communicate_DUT_IO, Profiled_Without_Clone, 1, High_SNR).

Countermeasure: Single trace attacks on the message encoding operation can also be concretely prevented by shuffling the message encoding operation, as proposed by Reference [3]. Shuffling ensures that the attacker can recover all the bits of the message, but not the order of message bits. This concretely prevents message recovery, since full shuffling has an entropy of $\binom{n}{k}$, where k is the Hamming weight of the message with $n = 256$ for Kyber KEM and k having an average

value of $k = 128$. Thus, $\binom{n}{k}$ is beyond brute-force for an attacker. We henceforth refer to this countermeasure using the label `Shuffled_Encode`.

Refer to Table 1 for a tabulation of all side-channel attacks on the encapsulation procedure of Kyber KEM.

3.4 SCA on Decapsulation Procedure in Ephemeral Key Setting

In this section, we discuss single trace side-channel attacks applicable to the decapsulation procedure in the ephemeral key setting. We recall that both message recovery and key recovery attacks are possible and that message recovery has the same impact as that of performing key recovery in the ephemeral key setting.

3.4.1 SASCA. The encapsulation procedure is also susceptible to SASCA-based attacks.

Targeting NTT in the Decryption Procedure (SASCA_NTT): An attacker can target the INTT operation over $\hat{g}'$, i.e., the product of NTT of ciphertext component $\mathbf{u}'$ and the NTT of secret $\mathbf{s}$ (Line 27 of Decrypt in Algorithm 1). Recovery of $\hat{g}'$ results in trivial recovery of the secret key $\mathbf{s}$, since the ciphertext component $\mathbf{u}$ is known to the attacker. The attack can be carried out using templates from a clone device. We thus define the characteristic of the attack using the following tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Targeting NTT in the Re-encryption Procedure (SASCA_NTT): Apart from targeting NTT in the decryption procedure, an attacker can also target NTT instances in the re-encryption procedure in the same manner as targeting NTT instances in the encapsulation procedure. In this respect, an attacker can target the NTT operation over the ephemeral secret $\mathbf{r}$ within the re-encryption procedure (Line 20 in Algorithm 2). This enables the attacker to recover $\mathbf{r}$, whose knowledge can be used to recover the message m for a given target ciphertext ct .

There is, however, one subtle difference with respect to profiling when compared to the same attack on the encapsulation procedure. If the attacker is able to communicate with the decapsulation procedure (Communicate_DUT_IO), then he/she can build templates using leakage from decapsulation of valid ciphertexts directly from the DUT.

This can be done in the following manner: The attacker can construct valid ciphertexts ct_i for $i \in [0, T - 1]$ for which the attacker knows the value of the ephemeral secret $\mathbf{r}_i$ for $i \in [0, T - 1]$. Leakage from decapsulation of these ciphertexts can be used to build templates for all internal operations within the NTT over $\mathbf{r}$. In this scenario, we define the attack characteristic using the tuple: (Communicate_DUT_IO, Profiled_Without_Clone, 1, High_SNR). However, if the attacker cannot communicate with the DUT, then he/she requires access to a clone device for profiling. In this scenario, we define the attack characteristic using the tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Targeting KECCAK after Decryption Procedure (SASCA_KECCAK): An attacker can target KECCAK instances after the decryption procedure, similar to the attack on KECCAK instances in the encapsulation procedure. One can target the KECCAK instance used as a PRF to generate the pre-key $\bar{K}'$ (Line 18 of Decaps in Algorithm 2) for message recovery. Similarly, the attacker can target the KECCAK used as a PRNG in the re-encryption procedure (Line 14 of Encrypt in Algorithm 1) to recover $\mathbf{r}$, leading to message recovery. It is important to note that both operations primarily depend upon the message m' . Thus, an attacker who can communicate with the DUT can control m' during valid key-exchanges and build templates directly on the DUT. In this scenario, we define the attack characteristic using the tuple: (Communicate_DUT_IO, Profiled_Without_Clone, 1, High_SNR). However, if the attacker cannot communicate with the DUT, then templates can only be built on the clone device, i.e., (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Countermeasure: Shuffled_Masked_NTT as well as Shuffled_KECCAK countermeasures can be used to protect against SASCA-style attacks.

3.4.2 Targeting Message Decoding. Similar to the message encoding operation within the Encrypt procedure, the message decoding operation within the decryption procedure (Decrypt) also performs bitwise manipulation of the decrypted message m' (Line 28 of Decrypt in Algorithm 1). The erroneous message polynomial $\mathbf{m}' \in R_q$ is decoded into the message $m' \in \mathcal{B}^{32}$ one coefficient at a time. This bitwise manipulation was shown to be exploitable by Ravi et al. [56] using single trace attacks on Kyber KEM. We refer to this attack using the label Message Decode. Subsequently, Ngo et al. [46] demonstrated a similar message recovery attack on the masked decoding procedure in Saber KEM, which has also been extended to higher-order masked implementations as well [48, 56]. We refer to this attack using the label Masked_Message_Decode.

If the attacker can communicate with the DUT (Communicate_DUT_IO), then templates for the message can be built directly on the DUT. Thus, the attack characteristic in this scenario is (Communicate_DUT_IO, Profiled_Without_Clone, 1, High_SNR). If the attacker cannot communicate with the DUT (Observe_DUT_IO), then templates have to be built on a clone device. Thus, the attack characteristic in this scenario is (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Similar to the Message_Encode attacks on the message encoding procedure, attack on the decoding procedure also requires less noise in the measurements. Moreover, leakage from manipulation of single message bits only span for a single clock cycle. Thus, adapting the attack to advanced platforms with inherent measurement or algorithmic noise is not very trivial.

Countermeasure: Similar to shuffling the message encoding procedure, shuffling the message decoding procedure provides concrete protection against such single-trace message recovery attacks, as proposed in Reference [3]. We henceforth refer to this countermeasure as Shuffled_Decode.

Apart from the aforementioned attacks, we observe that single attacks on the encapsulation procedure also apply to the decapsulation procedure. This is because all operations targeted by side-channel attacks on the encapsulation procedure are also performed in the decapsulation procedure due to the use of the FO transform (refer to Section 3.3).

Refer to Table 1 for a tabulation of all side-channel attacks on the decapsulation procedure in the ephemeral key setting of Kyber KEM.

3.5 SCA on Decapsulation Procedure in Static Key Setting

In the static key setting, the decapsulation procedure manipulates the same secret key sk for multiple key-exchanges. Thus, the attacker has access to multiple traces from the decapsulation procedure to perform key-recovery and message recovery. Clearly, single trace attacks applicable to the decapsulation procedure in the ephemeral key setting are also applicable in the static key setting (refer to Section 3.4). Thus, we only discuss those attacks that utilize multiple traces from the decapsulation procedure for key recovery.

3.5.1 Correlation Power Analysis (CPA). We first discuss attacks that assume an Observe_DUT_IO attacker, who can only passively monitor I/O of the DUT performing decapsulation. In this respect, Mudgei et al. [44] performed an extensive study on CPA-style attacks and their applicability to different polynomial multiplication strategies, including NTT. Kyber adopts an *incomplete* NTT for polynomial multiplication for efficiency reasons, i.e., it only computes $(\log_2(n) - 1)$ layers of the NTT for an $n - 1$ degree polynomial. Thus, the output of the incomplete NTT is nothing but a sequence of linear polynomials with degree 1. The decryption procedure computes the incomplete NTT of $\mathbf{u}' \in R_q^k$ (Line 25 in Algorithm 1), which is followed by a point-wise multiplication with the coefficients of the NTT transformed secret $\hat{\mathbf{s}} \in R_q^k$, i.e., $(\hat{\mathbf{u}}' \circ \hat{\mathbf{s}})$ in Line 26 of Algorithm 1.

This point-wise multiplication is performed using several 2-coefficient schoolbook multiplication operations.

Mujdei et al. [44] showed that leakage from the schoolbook polynomial multiplications after the incomplete NTT can be exploited through conventional CPA-style attacks. The presented attack is a non-profiled attack, similar to other CPA-style attacks, and required ≈ 200 power traces to recover all the coefficients of $\hat{s}$, which enables full key recovery. Similarly, Chen et al. [16] demonstrated a non-profiled CPA attack targeting the school-book polynomial multiplication over NTT transformed polynomials used in the signing procedure of Dilithium for key recovery. Their attack can also be adapted to Kyber, which requires less than 200 power traces for key recovery. We refer to these attacks using the label `NTT_Leakage_CPA`. Since these attacks work over multiple traces, they can still work with low SNR. The attack characteristic can be described using the following tuple: (Observe_DUT_IO, Non_Profiled, ≈ 200 , Low_SNR).

Countermeasure: Similar to typical CPA-style attacks, masking serves as a concrete countermeasure, which splits the sensitive variable into multiple shares and operates over each of them independently throughout the implementation. We refer to this countermeasure using the label `Masking` [13, 32]. However, one of the disadvantages of masking is its high cost ($\approx 2.5\text{--}3\times$), as clearly shown in References [13, 32].

In the following, we discuss those attacks that assume a `Communicate_DUT_IO` attacker, who can submit ciphertexts of his/her choice for decapsulation by the DUT. Several works have shown that such an attacker can exploit leakage from the decapsulation procedure in different ways to carry out key recovery attacks [56, 63, 76]. This forms the largest category of attacks applicable to lattice-based KEMs such as Kyber KEM, which we refer to as **side-channel-assisted chosen-ciphertext attacks (SCA assisted CCA)**.

3.6 Side-channel-assisted Chosen-ciphertext Attacks

Kyber KEM is IND-CCA secure and therefore enjoys concrete theoretical security guarantees against classical chosen-ciphertext attacks, which query the target with malformed/handcrafted chosen ciphertexts. This is primarily due to the attacker's inability to access any information about sensitive intermediate variables in the decapsulation procedure. However, an attacker who can utilize side-channel leakage can realize a practical *oracle* to obtain critical information about secret-dependent internal variables within the decapsulation procedure for chosen-ciphertexts, leading to key recovery.

In the following, we discuss the different types of SCA-assisted CCAs applicable to Kyber KEM. Their modus operandi is given as follows: The attacker queries the decapsulation procedure with handcrafted ciphertexts. These ciphertexts are crafted such that the decrypted message m' is very closely related to a targeted portion of the secret key, or in a few cases, the entire secret key. The attacker utilizes leakage from the decapsulation procedure to recover information about m' , thereby realizing a practical side-channel *oracle*. Such information obtained over several carefully crafted ciphertexts enables full key recovery. Following are the major sub-categories of side-channel oracle-based CCAs:

- (1) Binary Plaintext-Checking (PC) Oracle-based SCA
- (2) Parallel Plaintext-Checking (PC) Oracle-based SCA
- (3) Decryption-Failure (DF) Oracle-based SCA
- (4) Full-Decryption (FD) Oracle-based SCA

3.6.1 Binary Plaintext-Checking (PC) Oracle-based SCA. An attacker constructs ciphertexts to ensure that m' (Line 28 in Algorithm 1) only depends upon a single targeted coefficient of the

secret key. Side-channel leakage from the subsequent operations processing m' (Lines 18–20 in Algorithm 2) are used to instantiate a PC oracle for key recovery. We briefly explain the PC oracle-based SCA on Kyber KEM, and the same attack can also be adapted to other LWE/LWR-based schemes such as Saber, as shown in Reference [63]. Referring to Algorithm 1, the attacker chooses a very sparse ciphertext $ct = (\mathbf{u}, \mathbf{v}) \in (R_q^k \times R_q)$ as follows:

$$\mathbf{u}_i = \begin{cases} U \cdot x^0 & \text{if } i = 0, \\ 0 & \text{if } 1 \leq i \leq k - 1 \end{cases} \quad (4)$$

$$\mathbf{v} = V \cdot x^0, \quad (5)$$

where $(U, V) \in \mathbb{Z}^+$. For this chosen-ciphertext, each bit of the decrypted message m' , i.e., m'_i for $i \in [0, n - 1]$ is given as:

$$m'_i = \begin{cases} \text{Decode}(V - U \cdot s_0[0]), & \text{if } i = 0 \\ \text{Decode}(-U \cdot s_0[i]), & \text{for } 1 \leq i \leq n - 1. \end{cases} \quad (6)$$

Thus, every bit m'_i is only dependent on a single corresponding secret coefficient of s_0 , i.e., $s_0[i]$. The attacker can choose tuples (U, V) such that:

$$m'_i = \begin{cases} \mathcal{F}(s_0[0]), & \text{if } i = 0 \\ 0, & \text{for } 1 \leq i \leq n - 1. \end{cases} \quad (7)$$

Now, m' can only take two possible values, i.e., $m' = 0/1$, whose value depends upon a single secret coefficient $s_0[0]$. Thus, $m' = 0/1$ for different tuples (U, V) can be used as a binary distinguisher for every possible candidate of $s_0[0]$. Recovery of $m' = 0/1$ is done through side channels (side-channel-based PC oracle). In the similar manner, attacker can build ciphertexts to recover the other secret coefficients, one at a time, leading to full key recovery.

In this respect, D'Anvers et al. [21] presented the first SCA-assisted CCA, targeting a non-constant time implementation of LAC, a Ring-LWE-based KEM [39]. The targeted design utilized a non-constant time implementation of the BCH decoding procedure. D'Anvers et al. [21] showed that the time taken to decode $m' = 0$ after decryption is much smaller than time taken to decode $m' = 1$. This timing side-channel information was used to recover m' for chosen-ciphertexts, which resulted in key recovery in a few thousand chosen-ciphertexts for LAC KEM.

Subsequently, Ravi et al. [63] generalized the attack to several constant-time implementations of LWE/LWR-based KEMs, including Kyber KEM, utilizing the power/EM side-channel. They observed that a single-bit difference in m' (0/1) uniformly randomizes all subsequent operations after decryption, i.e., due to the use of hash functions in the decapsulation procedure (Lines 18–20 in Algorithm 2). Thus, power/EM side-channel leakage from any of these operations can be used to realize a practical binary PC oracle to distinguish between $m' = 0$ and $m' = 1$.

The attack works in two phases: (1) Pre-processing Phase and (2) Key Recovery Phase.

- (1) *Pre-processing Phase*: In this phase, side-channel templates are constructed for leakage from the re-encryption procedure for $m' = 0$ and $m' = 1$ using a simple Welch's t -test. Templates can be built directly on the DUT by querying with valid ciphertexts corresponding to $m' = 0$ and $m' = 1$. Since leakage from the re-encryption procedure depends upon both m' and pk , templates have to be built for every new key pair (pk, sk) .
- (2) *Key Recovery Phase*: In this phase, the attacker obtains single traces corresponding to all the chosen-ciphertexts (Equation (5)) and, subsequently, each attack trace is classified as either $m' = 0/1$ through simple template matching. This information is sufficient for full key recovery.

More recently, Ueno et al. [71] studied the applicability of the aforementioned attack to all KEMs in the NIST standardization process and demonstrated that almost all KEMs were susceptible to similar binary PC oracle-based chosen-ciphertext attacks.

One of the main advantages of the attack is that it can be carried out without any knowledge or very minimal knowledge about the implementation. Moreover, any operation after the decryption procedure (Lines 18–20 of Decaps in Algorithm 2) can be exploited to instantiate a practical PC oracle for key recovery, which amounts to a few hundred to few thousand leakage points. Thus, the attack can also work with low SNR due to a large number of leakage points available for exploitation. However, the attack only recovers a single bit of information about the secret key in each query. Thus, full key recovery requires a few thousand ($\approx 1k-3k$) chosen-ciphertext queries for Kyber KEM. More recently, few works have proposed improved methods to construct chosen-ciphertexts to reduce the number of queries for key recovery [8, 54] and also to perform efficient key recovery in the presence of a non-perfect side-channel binary PC oracle [66].

We therefore refer to the aforementioned attacks together using the label `Binary_PC_Oracle_CCA` attack. We define the attack characteristic using the following tuple: (`Communicate_DUT_IO`, `Profiled_Without_Clone`, $\approx 1k-3k$, `Low_SNR`).

Countermeasure: Masking the decapsulation procedure serves as a concrete countermeasure against the `Binary_PC_Oracle_CCA` attack (Masking [13, 32]). While higher-order attacks are still possible, they incur corresponding exponential increase in number of traces for key recovery.

3.6.2 Parallel Plaintext-Checking (PC) Oracle-based SCA. Very recently, Rajendran et al. [55] and Tanaka et al. [70] demonstrated improved PC oracle-based side-channel attacks, which are capable of more than one bit of information per query. They demonstrated the ability to recover a generic P number of bits of information about the secret key in a single query ($P \in \mathbb{Z}^+$) through construction of modified ciphertexts $ct = (\mathbf{u}, \mathbf{v}) \in (R_q^k \times R_q)$ as follows:

$$\mathbf{u}_i = \begin{cases} U \cdot x^0 & \text{if } i = 0, \\ 0 & \text{if } 1 \leq i \leq k-1 \end{cases} \quad (8)$$

$$\mathbf{v} = V \cdot \left(\sum_{i=0}^{i=(P-1)} x^i \right), \quad (9)$$

where $(U, V) \in \mathbb{Z}^+$. For this chosen-ciphertext, each bit of the decrypted message m' , i.e., m'_i for $i \in [0, n-1]$, is given as:

$$m'_i = \begin{cases} \text{Decode}(V - U \cdot s_0[i]), & \text{for } i \in [0, P-1] \\ \text{Decode}(-U \cdot s_0[i]), & \text{for } P \leq i \leq n-1. \end{cases} \quad (10)$$

Thus, every bit m'_i for $i \in [0, P-1]$ is only dependent on a single corresponding secret coefficient of s_0 , i.e., $s_0[i]$. The attacker can choose tuples (U, V) such that:

$$m'_i = \begin{cases} \mathcal{F}(s_0[i]), & \text{for } i \in [0, P-1] \\ 0, & \text{for } P \leq i \leq n-1. \end{cases} \quad (11)$$

Thus, the first P bits of m' , i.e., m'_i for $i \in [0, P-1]$, are now dependent on the corresponding coefficients of s_0 , i.e., $s_0[i]$ for $i \in [0, P-1]$, while all the other bits are fixed to 0. Thus, each of the P message bits serve as a binary distinguisher for the corresponding coefficient of s_0 . An attacker who can recover these P message bits per query can realize a P -way parallel PC oracle for key recovery. We refer to it as the `Parallel_PC_Oracle_CCA` attack.

The realization of such a P -way parallel PC oracle reduces the number of attack traces/queries for key recovery by a factor of P , compared to the Binary_PC_Oracle_CCA attack [21, 63]. In this respect, Rajendran et al. [55] and Tanaka et al. [70] experimentally demonstrate that there is enough information present in power/EM side-channel leakage from the re-encryption procedure to distinguish between 2^P possible values of the message m' in a single trace for $P < 10$. For $P = 10$, full key recovery can be done in ≈ 200 traces. However, higher values for P , if achievable, can further reduce the number of attack traces for key recovery.

However, it is important to note that increasing P also exponentially increases the number of templates to be built in the pre-processing phase (2^P), while the number of traces in the attack phase only reduces linearly by a factor of P . If an attacker has access to a clone device (Profiled_WithClone), then template phase can be completely taken offline, allowing to arbitrarily increase P to reduce the number of queries to the DUT. However, if there is no clone device access, then attacker has to identify a tradeoff between traces for the pre-processing phase and the key recovery phase. We therefore define the characteristic of Parallel_PC_Oracle_CCA attack using the following tuples: (Communicate_DUT_IO, Profiled_Without_Clone, ≈ 300 –500, Low_SNR), (Communicate_DUT_IO, Profiled_WithClone, ≈ 100 –200, Low_SNR).

Countermeasure: Similar to the Binary_PC_Oracle_CCA attack, masking the entire decapsulation procedure serves as a concrete countermeasure against the Parallel_PC_Oracle_CCA attack (Masking [13, 32]).

3.6.3 Decryption-Failure (DF) Oracle-based SCA. This category of attacks work by querying the decapsulation device with carefully perturbed ciphertexts, such that the decryption failures in the decrypted message m' depend upon the secret key. A side-channel oracle that is able to detect decryption failures can therefore recover the secret key.

The core idea of the attack is as follows: The attacker generates a valid ciphertext $ct = (\mathbf{u}, \mathbf{v})$ for a message m and adds single coefficient errors to the second component $\mathbf{v}$, e.g., $\bar{\mathbf{v}} = \mathbf{v} + e \cdot \mathbf{x}^0$ (adding error to the first coefficient), where $e \in \mathbb{Z}^+$. This has the effect of perturbing the first coefficient of the erroneous message polynomial, i.e., $m'[0]$ by e (Line 27 of Decrypt in Algorithm 1), thereby increasing the first coefficient of the noise component $\mathbf{d}$, i.e., $\mathbf{d}[0]$ by e (refer to Equation (3)). If the error e is large enough to push $m'[0]$ beyond $q/4$ (respectively, $3q/4$) for $m'_0 = 0$ (respectively, $m'_0 = 1$), then this flips m'_0 , resulting in a decryption failure.

The size of e that triggers a decryption failure provides information about the original noise $\mathbf{d}[0]$, which is linearly dependent on the secret $\mathbf{s}$ (Equation (3)). Thus, an attacker who can obtain such information over several chosen-ciphertexts can recover the full secret key [11, 29].

The first such attack exploiting a side-channel-based DF oracle was proposed by Guo et al. [29] on Frodo KEM. They demonstrated that decryption failure can be detected through side-channel leakage from the ciphertext comparison operation (Line 21 in Algorithm 2). The key observation is that the re-computed ciphertext ct_R solely depends upon the decrypted message m' (Line 20 in Algorithm 2). Even a single bit change in m' results in a completely different recomputed ciphertext ct_R (due to the use of hash function). Thus, for a perturbed ciphertext ct that does not lead to a decryption failure, the ciphertext comparison only fails for a single coefficient of $\mathbf{v}$, while all other coefficients of both $\mathbf{u}$ and $\mathbf{v}$ match correctly with that of the recomputed ciphertext. However, in case of a decryption failure, the coefficients of ct_R are completely random, which ensures that the ciphertext comparison fails in multiple coefficients with an overwhelming probability.

In this respect, Guo et al. [29] targeted the implementation of Frodo KEM, which utilizes a non-constant time comparison of the ciphertext comparison operation and exploited the difference in comparison time to instantiate a practical DF oracle for full key recovery. Subsequently, Bhasin et al. [11] adapted the attack, which exploits power/EM side-channel leakage from constant-time

implementations of the ciphertext comparison operation. They identified flaws in common approaches used for masking the ciphertext comparison operation proposed in References [7, 49]. Subsequent works have proposed secure masking schemes for the ciphertext comparison operation used in lattice-based KEMs [11, 18]. We refer to these attacks using the label DF_Oracle_CCA attack. They have similar attack Characteristics (plural) as that of the Binary_PC_Oracle_CCA attack, i.e., (Communicate_DUT_IO, Profiled_Without_Clone, $\approx 5k-6k$, Low_SNR), while consuming slightly more number of traces for key recovery, compared to the Binary_PC_Oracle_CCA attack.

Countermeasure: Masking the entire decapsulation procedure serves as a concrete countermeasure against the DF_Oracle_CCA attack (Masking [13, 32]).

3.6.4 Full-Decryption (FD) Oracle-based SCA. The aforementioned PC_Oracle_CCA and DF_Oracle_CCA attacks work by recovering anywhere between 1 to P bits of information about the secret key ($P \in \mathbb{Z}^+$) from a single chosen-ciphertext query. This gives rise to a natural question: whether it is possible to recover the entire message m' in a single query for chosen-ciphertexts. In this respect, Xu et al. [76] proposed that operations that leak the complete message, exploited by message recovery attacks for valid ciphertexts, can also be exploited in a chosen-ciphertext setting to realize an FD oracle. In this manner, an attacker can recover 256 bits of information about the secret key s in a single trace.

To realize an FD oracle, they propose to construct ciphertexts $ct = (\mathbf{u}, \mathbf{v}) \in (R_q^k \times R_q)$ such that

$$\mathbf{u}_i = \begin{cases} U \cdot x^0 & \text{if } i = 0, \\ 0 & \text{if } 1 \leq i \leq k-1 \end{cases} \quad (12)$$

$$\mathbf{v} = V \cdot \left(\sum_{i=0}^{i=n-1} x^i \right), \quad (13)$$

where $(U, V) \in \mathbb{Z}^+$. The attacker can choose tuples (U, V) such that the decrypted message is nothing but

$$m'_i = \{ \mathcal{F}(s_0[i]), \text{ if } 0 \leq i \leq n-1, \} \quad (14)$$

where every message bit m'_i is dependent upon the corresponding secret coefficient of s_0 , i.e., $s_0[i]$. Moreover, the attacker can choose (U, V) such that every message bit m'_i uniquely identifies the corresponding secret coefficient $s_0[i]$ for $i \in [0, 255]$. To realize a practical FD oracle, Xu et al. [76] proposed to exploit leakage from the message encoding operation during re-encryption (Line 20 in Algorithm 1), which enables to recover the entire message in a single trace. Thus, full key recovery is possible in only six queries for Kyber512.

Similarly, Ravi et al. [56] and Ngo et al. [46, 47] showed that leakage from the message decoding operation (Line 28 in Algorithm 1), i.e., Message_Decode attack, can also be exploited in a chosen-ciphertext setting for key recovery in approximately 6–20 traces from schemes such as Kyber and Saber. We refer to these attacks using the label FD_Encode_Decode_Oracle_CCA. We define the attack characteristic using the tuple: (Communicate_DUT_IO, Profiled_Without_Clone, $\approx 6-20$, High_SNR).

Apart from attacks exploiting the power/EM side-channel, a few recent works have demonstrated FD oracle-based key recovery attacks that exploit far-field amplitude modulated EM emanations from on-board antennas on mixed-signal chips [73, 74]. This side-channel can work over longer distances compared to the EM side-channel but inherently contain more background noise, thereby increasing the number of traces for key recovery.

Targeting Protected Implementations of Message Encoding/Decoding Operation: We recall that, in the presence of an Observe_DUT_IO attacker, Message_Encode and Message_Decode attacks targeting the message encoding and decoding procedures for message recovery can be thwarted using the shuffling countermeasure (Shuffled_Encode, Shuffled_Decode). However, in the presence of a Communicate_DUT_IO attacker, when targeting the decapsulation procedure in a static key setting, Ravi et al. [56] showed that the shuffling countermeasures can be broken, exploiting the *ciphertext malleability* property of LWE/LWR-based schemes.

We briefly describe their attack exploiting leakage from the shuffled encoding operation, which recovers the message one bit at a time. The shuffling countermeasure does not remove leakage, but only ensures that the shuffling order of the message bits cannot be recovered by the attacker. Given a target ciphertext $ct = (u, v)$ whose message is to be recovered, the attacker first submits the target ciphertext ct to the decapsulation procedure and recovers the individual message bits of m' through side channels and subsequently computes its **Hamming Weight (HW)**. Subsequently, the attacker submits a perturbed ciphertext $ct' = (u, v + q/2 \cdot x^0)$, i.e., $q/2$ added to the first coefficient of v . This has the effect of flipping the first message bit m'_0 , resulting in a perturbed message m'' . If $HW(m'') = HW(m') - 1$, then the perturbation flipped m'_0 from 1 to 0, thus deducing that $m'_0 = 1$. Otherwise, if $HW(m'') = HW(m') + 1$, then $m'_0 = 0$. In this manner, an attacker can induce bit-flips in all the 256 bits of the message to completely recover the message in 257 queries for Kyber KEM. Thus, shuffling increases the attacker's effort from recovering 256 bits in a single trace to recovering 1 bit per trace. Nevertheless, shuffling does not concretely prevent message recovery and key recovery in a chosen-ciphertext setting. Extending upon this idea, recently Ngo et al. [47] demonstrated improved attacks to break the combined shuffling and masking countermeasure for the message decoding operation in Saber.

We can clearly observe that an attacker with the Communicate_DUT_IO capability can perform improved attacks to break countermeasures such as shuffling, which are otherwise considered secure in the presence of an Observe_DUT_IO attacker. We refer to these attacks targeting the shuffled encoding/decoding procedure using the label Shuffled_Encode_Decode_FD_Oracle_CCA, and their characteristic tuple is (Communicate_DUT_IO, Profiled_Without_Clone, $\approx 2k-3k$, High_SNR). The attack on the masked encoding/decoding procedure is denoted using the label Masked_Encode_Decode_FD_Oracle_CCA and its characteristic tuple is (Communicate_DUT_IO, Profiled_Without_Clone, $\approx 10-20$, High_SNR). The attack on the shuffled and masked encoding/decoding procedure using the label Shuffled_Masked_Encode_Decode_FD_Oracle_CCA. We define the attack characteristic using the tuple: (Communicate_DUT_IO, Profiled_Without_Clone, $\approx 2k-3k$, High_SNR).

Countermeasure: As shown above, shuffled and masked implementations of the message encoding and decoding procedures do not prevent realization of an FD oracle for key recovery [46, 47, 56]. Since leakage from the message encoding/decoding procedure spans for only 1 to a few clock cycles for each message bit, addition of jitter serves as a reasonable mitigation technique, but it does not concretely prevent the attack. Thus, increasing the key refreshment rate to repeatedly change the public key serves as the only strong countermeasure against the attack. This ensures that the attacker cannot obtain enough traces from the decapsulation procedure to recover a single secret key. However, the exact key refresh rate required to prevent these attacks depends upon the DUT and the attack setup.

3.6.5 Targeting NTT in a CCA Setting. While leakage from the INTT instance over $\hat{g}' = (\hat{u}' \circ \hat{s})$ in the decryption procedure has been exploited for key recovery in the Observe_DUT_IO setting (Line 27 of Decrypt in Algorithm 1), the attack relies on extremely low-noise measurements for successful key recovery (SASCA_NTT attack [51, 53]). The authors show that the attack can tolerate a

Table 1. Tabulation of Reported SCA and Their Characteristics for the Different Procedures of Kyber KEM

Attack	Attack Characteristic					
	Attack_Vector	DUT_IO_Access	Profile_Requirement	No_Traces	SNR	Countermeasure
Key Generation						
SASCA_NTT [51, 53]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Masked_NTT
SASCA_KECCAK [36]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_KECCAK
Encapsulation						
SASCA_NTT [51, 53]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Masked_NTT
SASCA_KECCAK [36]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_KECCAK
Message_Encode [3, 67, 76]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Encode
		Communicate_DUT_IO	Profiled_Without_Clone	1	High_SNR	Shuffled_Encode
Masked_Message_Encode [46, 48]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Encode
		Communicate_DUT_IO	Profiled_Without_Clone	1	High_SNR	Shuffled_Encode
Decapsulation (Ephemeral Key)						
SASCA_NTT [51, 53]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Masked_NTT
		Communicate_DUT_IO	Profiled_Without_Clone	1	High_SNR	Shuffled_Masked_NTT
SASCA_KECCAK [36]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_KECCAK
		Communicate_DUT_IO	Profiled_Without_Clone	1	High_SNR	Shuffled_KECCAK
Message_Decode [56]	Power/EM	Observe_DUT_IO	Profiled_Without_Clone	1	High_SNR	Shuffled_Decode
		Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Decode
Masked_Message_Decode [46, 48]	Power/EM	Communicate_DUT_IO	Profiled_Without_Clone	1	High_SNR	Shuffled_Decode
		Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Decode
Decapsulation (Static Key)						
CT_Sanity_Check	Power/EM	Observe_DUT_IO	Non_Profiled	≈ 200	Low_SNR	Masking
Message_Poly_Sanity_Check						
NTT_Leakage_CPA [16, 44]						
CCA_SASCA_NTT [30]	Power/EM	Communicate_DUT_IO	Profiled_With_Clone	2–4	Low_SNR	Shuffled_Masked_NTT, CT_Sanity_Check, Message_Poly_Sanity_Check
Binary_PC_Oracle_CCA [21, 63, 66]	Power/EM [63, 66], Timing [21]	Communicate_DUT_IO	Profiled_Without_Clone	≈ 2k–3k	Low_SNR	Masking, CT_Sanity_Check, Message_Poly_Sanity_Check
Parallel_PC_Oracle_CCA [55, 70]	Power/EM	Communicate_DUT_IO	Profiled_Without_Clone	≈ 100–200	Low_SNR	Masking, CT_Sanity_Check, Message_Poly_Sanity_Check
		Communicate_DUT_IO	Profiled_With_Clone	≈ 300–500	Low_SNR	Masking, CT_Sanity_Check, Message_Poly_Sanity_Check
DF_Oracle_CCA [11, 18, 29]	Power/EM [11, 18], Timing [29]	Communicate_DUT_IO	Profiled_Without_Clone	5k–7k	Low_SNR	Masking, CT_Sanity_Check, Message_Poly_Sanity_Check
FD_Oracle_CCA [56, 73, 74, 76]	Power/EM [56, 76], Modulated EM [73, 74]	Communicate_DUT_IO	Profiled_Without_Clone	6–20	High_SNR	CT_Sanity_Check, Message_Poly_Sanity_Check
Masked_FD_Oracle_CCA [46, 48]	Power/EM	Communicate_DUT_IO	Profiled_Without_Clone	6–20	High_SNR	CT_Sanity_Check, Message_Poly_Sanity_Check
Shuffled_FD_Oracle_CCA [56]	Power/EM	Communicate_DUT_IO	Profiled_Without_Clone	≈ 1k–3k	High_SNR	CT_Sanity_Check, Message_Poly_Sanity_Check
Shuffled_Masked_FD_Oracle_CCA [47]	Power/EM	Communicate_DUT_IO	Profiled_Without_Clone	≈ 1k–3k	High_SNR	CT_Sanity_Check, Message_Poly_Sanity_Check

noise with standard deviation σ in the range 0.5–0.7. Recently, Hamburg et al. [30] demonstrated that the sensitivity of these attacks to SNR can be significantly improved in a chosen-ciphertext setting (Observe_DUT_IO). Their idea was to craft chosen-ciphertexts such that coefficients of $\hat{g}'$ is sparse and that leakage from the NTT operation over $\hat{g}'$ reportedly improves the effectiveness of the BP algorithm by allowing more noise in the measurements, even when targeting masked implementations. They demonstrate a range of key recovery attacks with trace complexity ranging from k to $2k$, where k is the dimension of the module in Kyber KEM ($k = \{2, 3, 4\}$). The improved attack can tolerate much more noise with standard deviation $\sigma \leq 2.2$, thereby demonstrating significant improvement in SASCA_NTT attacks when performed in a chosen-ciphertext setting. We refer to the attacks targeting the NTT using the label CCA_SASCA_NTT attack. We define the attack characteristic using the tuple: (Communicate_DUT_IO, Profiled_With_Clone, 2–4, High_SNR).

Countermeasure: Shuffling or masking the NTT operation as proposed by Ravi et al. [61] provides concrete protection against SASCA-style attacks.

Refer to Table 1 for a tabulation of all side-channel attacks on the decapsulation procedure in the static key setting of Kyber KEM.

3.7 Protection against SCA-assisted CCA

We observe that SCA-assisted CCA form the largest category of attacks on Kyber KEM. Moreover, an attacker capable of querying the decapsulation device with chosen-ciphertexts can perform a variety of key recovery attacks, also capable of defeating certain masking and shuffling countermeasures [46–48, 56], with incremental increase in attacker’s effort compared to breaking

unprotected implementations. Moreover, it is not clear which order of masking protection is required to achieve security in a given setting, especially given that the cost of masking significantly increases with order of protection.

In this respect, we particularly focus on SCA-assisted CCA attacks that work with malicious ciphertexts and present detection-based countermeasures, which test whether a received ciphertext is malicious. If detected as malicious, then the DUT can simply reject the ciphertext and change/refresh the public-private key pair by re-running the key-generation procedure. This ensures that, upon detection, further exposure of the secret key is prevented. In the following, we propose two *detection* countermeasures against the proposed CCAs for Kyber KEM.

3.7.1 Ciphertext Sanity Check. The main idea of this countermeasure stems from the observation that ciphertexts used for the Binary_PC_Oracle, Parallel_PC_Oracle, FD_Oracle and CCA_SASCA_NTT attacks are very sparse with several zero coefficients (refer to Equations (5), (9), and (13) for the chosen-ciphertexts). However, the coefficients of a valid ciphertext are uniformly distributed in the range $[0, q]$, given that both ciphertext components are essentially LWE instances. This skew in the chosen-ciphertexts can be easily detected and flagged as malicious ciphertexts before it can be decapsulated. While this countermeasure was also proposed by Xu et al. [76] to protect against attacks utilizing skewed ciphertexts, a concrete mathematical analysis and implementation of the same is not presented.

Detection Technique: To detect the skew in the ciphertexts, we chose to utilize the mean and standard deviation of the ciphertext coefficients. For a given polynomial $\mathbf{x} \in R_q$, we denote the mean (μ) and standard deviation (σ) of the coefficients of $\mathbf{x}$ as $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$, respectively. We performed empirical simulations to calculate the mean and standard deviation of $\mu(\mathbf{u})$ and $\sigma(\mathbf{u})$ for single polynomials of the ciphertext component $\mathbf{u}$, as well as $\mu(\mathbf{v})$ and $\sigma(\mathbf{v})$ for the ciphertext component $\mathbf{v}$, corresponding to valid ciphertexts of Kyber KEM. Refer below for the obtained values for the mean and standard deviation for all four of the statistical metrics for Kyber KEM.

$$\begin{aligned} (\mu(\mu(\mathbf{u})), \sigma(\mu(\mathbf{u}))) &= (1663, 60) & (\mu(\mu(\mathbf{v})), \sigma(\mu(\mathbf{v}))) &= (1560, 60) \\ (\mu(\sigma(\mathbf{u})), \sigma(\sigma(\mathbf{u}))) &= (959, 27) & (\mu(\sigma(\mathbf{v})), \sigma(\sigma(\mathbf{v}))) &= (957, 27). \end{aligned} \quad (15) \quad (16)$$

Based on the standard deviation σ for each of these metrics, the designer can choose an acceptable range for each of these four metrics. For example, if a tail length of $(6 \cdot \sigma)$ is chosen, then the acceptable range for $\mu(\mathbf{u})$ is $[\mu(\mu(\mathbf{u})) - 6 \cdot \sigma, \mu(\mu(\mathbf{u})) + 6 \cdot \sigma]$. Smaller the acceptable range, higher is the possibility of false positives, i.e., detecting a valid ciphertext as malicious. However, a large acceptable range increases the chances of false negatives, thereby resulting in acceptance of skewed malicious ciphertext as valid.

Evaluation: We deduced through empirical simulations that a tail of length (6σ) for both mean and standard deviation leads to a probability of $\approx 2^{-22}$ for rejection of a valid ciphertext. The rejection is done solely based on analyzing the size of the ciphertext coefficients, and this does not have any relation to the secret key. It is therefore trivial to observe that a false positive only hampers the performance of the scheme, but does not provide any additional information about the secret key. The implementer/designer can choose an appropriate range based on the tolerance to allow false positives and rejection of valid ciphertexts. We henceforth refer to this as the CT_Sanity_Check countermeasure in this article. One can also include other kinds of checks, such as checking the number of zero coefficients in the received ciphertext as well as the decrypted message m' , which can enhance confidence in the detection mechanism.

While this countermeasure is capable of detecting skewed ciphertexts, chosen-ciphertexts used in the DF_Oracle_CCA attack [11, 29] contain uniformly random coefficients. Thus, the

DF_Oracle_CCA attack can bypass our CT_Sanity_Check countermeasure. In the following, we propose a novel countermeasure that is also capable of defeating CCA utilizing chosen ciphertexts with uniformly random coefficients.

3.7.2 Message Polynomial Sanity Check. This countermeasure relies on analyzing the coefficients of the noisy message polynomial $\mathbf{m}' = (\mathbf{v}' - \mathbf{u}' \cdot \mathbf{s})$ obtained during decryption of the received ciphertext ct (Line 27 in Algorithm 1). For valid ciphertexts, we observe that the coefficients of the $\mathbf{m}'$ are distributed according to a very narrow Gaussian distribution near $q/2$ or 0, i.e., $\mathbf{m}[i] = q/2 \pm \delta$ for $m_i = 1$ and $\mathbf{m}[i] = 0 \pm \delta$ for $m_i = 0$

$$\mathbf{m}[i] = \begin{cases} q/2 \pm \delta & \text{if } m_i = 1, \\ 0 \pm \delta & \text{if } m_i = 0, \end{cases} \quad (17)$$

where $\delta \ll q \in \mathbb{Z}^+$. The span δ depends upon the distribution of the noise component $\mathbf{d}$ (Equation (3)). We performed empirical simulations to deduce the distribution of the coefficients of the noise component $\mathbf{d}$. They follow a Gaussian distribution with a standard deviation $\sigma = 79$, around 0 and $q/2$.

However, we observe that the distribution of the coefficients of $\mathbf{m}'$ is not maintained in case of the DF_Oracle-based CCA attack. We observe that the DF_Oracle_CCA attack works by pushing one of the coefficients of $\mathbf{m}'$ ($\mathbf{m}'[i]$) to cross the $q/4$ threshold. This ensures that at least one message polynomial, i.e., $\mathbf{m}'[i]$ is not within the expected range, corresponding to that of a valid ciphertext. This also applies to the following attacks that utilize malicious/handcrafted ciphertexts: CCA_SASCA_NTT [30], Binary_PC_Oracle_CCA [21, 63, 66], Parallel_PC_Oracle_CCA [55, 70], DF_Oracle_CCA [11, 18, 29], FD_Oracle_CCA [56, 73, 74, 76] Masked_FD_Oracle_CCA [46, 48], Shuffled_FD_Oracle_CCA [56], Shuffled_Masked_FD_Oracle_CCA [47].

Detection Technique: Based on the aforementioned observation, we propose to test the distribution of the message polynomial coefficients for the received ciphertext. Let the acceptable range be $(q/2 \pm L \cdot \sigma)$ and $(0 \pm L \cdot \sigma)$, where $L \in \mathbb{Z}^+$ is left to the designer's choice. Larger the acceptable range $L \cdot \sigma$, smaller is the probability of flagging a valid ciphertext (false positive). However, choosing a smaller range raises the chances of missing detection of a malicious chosen-ciphertext. Thus, it is important to choose a conservative value for L for improved security. We performed experimental simulations for $L = 6$, and we were not able to observe a false positive for more than 2^{25} valid decapsulations. Once an invalid ciphertext is detected, the corresponding secret key is discarded and a new one needs to be generated, for reasons that will be explained below.

Evaluation: We subsequently tested several existing side-channel attacks [55, 56, 63] and found that for all attack ciphertexts used in these attacks there was a significant probability of triggering the countermeasure and thus discarding the secret key. More specifically, these attacks focus on one coefficient of the secret, and for all attack ciphertexts at least one possible value of this coefficient of the secret leads to detection of the attack. The attack of Rajendran et al. [55] also includes an attack that targets multiple coefficients at once, but this improvement only increases the probability of triggering the countermeasure. We did not find parameter sets that reliably avoided our countermeasure. Thus, we can conclude that for these attacks our countermeasure effectively restricts the number of useful invalid ciphertexts an attacker can input before the countermeasure is triggered and the secret key is discarded. The countermeasure would also effectively stop the attack described by Bhasin et al. [11]. This attack relies on finding the boundary where the message bit is flipped, but due to the countermeasure the region around the boundary results in the detection of the invalid ciphertext and the discard of the secret. Note that for $L = 6$ the discard

region has approximately the same size as the accept region, making it infeasible to add an error to push the ciphertext towards the boundary without triggering the discard.

In-depth Analysis: The increased decryption failure probability makes the scheme more vulnerable to decryption failure attacks [19]. To mitigate this, we only allow the adversary to obtain at most one failing ciphertext due to our countermeasure: If there is at least one coefficient outside this acceptable range, then we flag the ciphertext as invalid, discard the old public-private key pair, and generate a new public-private key pair.

Allowing the adversary to obtain one failing ciphertext does not significantly impact security in this scenario. As can be seen from References [17, 19, 20] one failing ciphertext is not enough to significantly reduce the security of the key pair, and the ciphertext is discarded after one failure caused by our countermeasure. Moreover, as the decryption failure probability is enlarged, the information in the decryption failure is reduced, as discussed in Reference [19]. This means that the leaked information from one failing ciphertext will be even smaller than in regular failure boosting attacks.

More in-depth, there are two scenarios to consider: First, the ciphertext is not accepted by the countermeasure, in which case the adversary has one failing ciphertext that, as discussed previously, does not significantly reduce the security of the public-private key pair. The key pair is subsequently discarded and, as such, the adversary can not gain additional information. Second, the ciphertext is accepted by the countermeasure, in which case there is no difference with the regular security framework of Kyber.

For a side-channel attacker, we observe that this countermeasure requires to decrypt at least one chosen-ciphertext for successful detection, however, the CCAs in interest require at least a few tens to few thousand queries for key recovery. Thus, we argue that allowing a single decapsulation of the chosen-ciphertext is not useful for the attacker. We henceforth refer to this countermeasure as `Message_Poly_Sanity_Check` throughout this article. As we show later in Section 4, this countermeasure can serve as a countermeasure for fault-assisted chosen-ciphertext attacks on Kyber KEM as well.

Countermeasures: The aforementioned detection countermeasures (`Message_Poly_Sanity_Check` and `CT_Sanity_Check`) can be specifically used to protect against attacks against the decapsulation procedure in the chosen-ciphertext setting. As we show later in Section 7.2, these countermeasures incur very less additional runtime compared to masking countermeasures for the decapsulation procedure. Thus, these countermeasures can be implemented as an add-on, on top of masked implementations of the decapsulation procedure. On the flip side, these countermeasures can only detect invalid/malicious ciphertexts, while they cannot deter attacks that work against CPA-style attacks (`NTT_Leakage_CPA`), which work with valid ciphertexts.

4 FAULT-INJECTION ATTACKS ON KYBER KEM

In this section, we discuss reported fault attacks on Kyber KEM. For every FIA discussed in this article, we also describe its characteristics based on the following parameters:

- (1) *Fault Injection Technique* (`Attack_Vector`): This characteristic denotes the type of fault injection technique used to carry out the attack: (1) **Voltage/Clock Glitching** (Glitching), (2) **Laser Fault Injection** (LFI), and (3) **Electromagnetic Fault Injection** (EMFI).
- (2) *Attacker's ability to communicate with DUT* (`DUT_IO_Access`): In this respect, we identify two categories: `Observe_DUT_IO`, `Communicate_DUT_IO`. Please refer to Section 3.1.4 for the description of these categories.
- (3) *Targeted or Non-targeted Fault* (`Targeted_Or_Not`): In this respect, we identify two categories:

- (a) **Targeted_Fault**: The attack works by injection faults to target specific variables or instructions, requiring to inject faults at precise instance in time.
- (b) **Non_Targeted_Fault**: The attack does not require injection of precise faults and can work with random perturbations to the target computation. Thus, precise time synchronization is not required.
- (4) *Number of Faults within Single Computation* (Num_Faults): This characteristic denotes the number of faults to be injected within a single execution of the target procedure.
- (5) *Total number of Faulty Computations* (Num_Executions): This indicates the total number of faulty computations/executions to recover the target secret variable. The number of executions is specified assuming that the expected fault is observed in every targeted execution of the computation. However, the exact number of faults required depends upon the design and the target platform.

Similar to SCA attacks on Kyber, we define the characteristic of each FIA on Kyber presented in the article using the following tuple: (Injection_Technique, DUT_IO_Access, Targeted_Or_Not, Num_Faults, Num_Executions). To explain the different attacks, we utilize the algorithm of IND-CPA secure PKE of Kyber in Algorithm 1 and algorithm of IND-CCA secure Kyber KEM in Algorithm 2. We also refer the reader to Figure 2 for an example key-exchange protocol that can be built using IND-CCA-secure Kyber KEM.

4.1 FIA on Key Generation

The key generation procedure serves as an attractive target for an attacker, particularly in an ephemeral key setting, since it is performed for every new key exchange. Injection of faults in the key-generation procedure could lead to faulty public keys that could easily compromise the secret key.

4.1.1 Targeting Sampling of Secrets. In this respect, Ravi et al. [62] proposed the first practical fault attack targeting the sampling of secrets and errors to generate LWE instances. Their attack stems from the observation that the seed used to sample the secret s and errors e only differ by a single byte, i.e., $seed_B$ appended by single-byte nonces $coins_s$ and $coins_e$ (Lines 5–6 of KeyGen in Algorithm 1). Thus, the attacker can use faults to force nonce reuse, i.e., $coins_s = coins_e$. This creates LWE instances of the form $t = A \cdot s + s = A \cdot (s + 1)$, which can be trivially solved using Gaussian elimination. Thus, the faulty public keys can be directly solved to recover the secret key. The faulty public keys are still valid to be used for valid key-exchange, and the injected faults have only reduced the entropy of the secret key. The authors demonstrated practicality of nonce-reuse using **Electromagnetic Fault Injection (EMFI)** on the ARM Cortex-M4 microcontroller. The attack requires to inject multiple targeted faults on the nonces used during the sampling procedure (1–10), depending upon the target scheme to attack, for full key recovery. We refer to this attack using the label **Nonce_Fault** attack. We describe the attack characteristic using the following tuple: (EMFI, Observe_DUT_IO, Targeted_Fault, 4–8, 1).

Countermeasure (Our Proposal): The basic idea of our countermeasure is to check if any two polynomials in the modules s and e are equal. If they are found to be equal, then it indicates that a fault has been injected with a very high probability. Subsequently, a random value is generated as the public key, which cannot be used for valid key exchanges. Thus, the injection of a successful fault on the nonce simply results in a **Denial of Service (DoS)** attack but does not result in compromise of the secret key.

Our countermeasure is implemented on two levels: inner level and outer level. The inner level comprises a verification procedure that checks if any two polynomials of s and e are equal. We denote this as the **Verify_Equality** procedure. However, one can argue that this procedure can

be bypassed by simply skipping the `Verify_Equality` procedure. Such double fault attacks can be prevented by carefully designing a loop counter. Thus, we propose an outer level protection that incorporates a `Dynamic_Loop_Counter` procedure to protect the `Verify_Equality` procedure to prevent trivial double fault attacks. The aforementioned two-level protection strategy of combining the `Verify_Equality` procedure and `Dynamic_Loop_Counter` procedure is referred to as the `Verify_Nonce_Fault` countermeasure. Please refer to Algorithm 5 for the pseudo-code of the `Verify_Nonce_Fault` protection. We briefly explain the `Verify_Equality` procedure followed by the `Dynamic_Loop_Counter` procedure.

Verify_Equality: First, polynomials within the same module $\mathbf{s} \in R_q^k$ and $\mathbf{e} \in R_q^k$ are checked for equality, i.e., comparison between $\mathbf{s}[i]$ (respectively, $\mathbf{e}[i]$) and $\mathbf{s}[j]$ (respectively, $\mathbf{e}[j]$) for $i \in [0, k-1]$, $j \in [0, k-1]$ and $i \neq j$. Subsequently, corresponding polynomials of $\mathbf{s}$ and $\mathbf{e}$ are also compared for equality, i.e., $\mathbf{s}[i]$ and $\mathbf{e}[i]$ for $i \in [0, k-1]$, $j \in [0, k-1]$ and $i = j$.

However, instead of comparing all $n = 256$ coefficients of each polynomial for equality, we propose to check only a random set of $X < n$ number of coefficients for equality. If a random set of X corresponding coefficients of two polynomials have the same value, then the two polynomials are considered to be equal. In this respect, X should be chosen to be large enough such that the probability of all X corresponding coefficients having the same value is very low. For Kyber768 with coefficients in $[-2, 2]$ (distributed based on CBD), the probability of $X = 10$ pairs of coefficients having the same value is $\approx 2 \cdot 10^{-6}$. This is the false positive rate for $X = 10$. The designer can choose appropriate value for X based on acceptable false positive rate.

The procedure to compare a given pair of polynomials is implemented as follows: First, a random value $rand \in \mathbb{Z}^+$ is sampled (Line 5 in Algorithm 5). Let the result of the verification procedure be denoted as `verify_result`, which is initialized as $rand$ (Line 6). For every pair of polynomials that are found to be equal, `verify_result` $\in \mathbb{Z}^+$ is incremented by $w \in \mathbb{Z}^+$. To incorporate redundancy, this check and increment can be done multiple times, say, $y \in \mathbb{Z}^+$ times. It is important to note that both w and y are sampled in random, for every execution. Thus, if a given pair of polynomials are found to be equal, then the value of `verify_result` is incremented by $y \cdot w$.

For $\mathbf{s} \in R_q^k$ and $\mathbf{e} \in R_q^k$, the total number of polynomial comparisons to be done is $n_p = \binom{k}{2} \cdot 2 + k^2$, i.e., considering comparison of corresponding polynomials in $\mathbf{s}$ and $\mathbf{e}$ as well as comparison of polynomials within $\mathbf{s}$ and $\mathbf{e}$ separately. Thus, the total number of coefficients compared is $C = n_p \cdot X$, where X is the total number of coefficients compared for a given pair of polynomials (chosen by designer). If no pair of polynomials are equal, then `verify_result` variable retains its original value $rand$, and this indicates success of verification, else the verification has failed. This entire procedure is implemented as `Verify_Equality` in Line 8 in Algorithm 5.

Dynamic_Loop_Counter: This outer level of protection is implemented to prevent trivial skipping attacks to bypass the `Verify_Equality` procedure. It is implemented in the following manner: The total number of coefficients of all polynomials of $\mathbf{s}$ and $\mathbf{e}$ is denoted as C . First, a random non-zero integer $g \in \mathbb{Z}^+$ is sampled (Line 3). Then, a loop counter lc is initialized to 0 and its value is increased by g (Line 2) for every coefficient that is compared within the `Verify_Equality` procedure. Thus, the final loop counter value lc is supposed to be $g \cdot C$ to indicate successful completion of the `Verify_Equality` procedure.

The public key pk is generated and stored in a temporary variable `temp`. It is copied one byte at a time to the actual output variable that is considered the public key pk_{out} (Line 12) only if the following conditions are satisfied (Line 11):

- (1) $lc = g \cdot C$, i.e., the loop counter lc is equal to the expected value $g \cdot C$, indicating completion of the `Verify_Equality` procedure.
- (2) `verify_result` $= rand$, i.e., indicating success of the `Verify_Equality` procedure.

ALGORITHM 5: Verify_Nonce_Fault countermeasure for KeyGen of Kyber KEM

```

1: procedure Verify_Nonce_Fault PROTECTED KeyGen
2:    $lc := 0$ 
3:    $g \in \mathbb{Z}^+ \leftarrow \text{Sample\_Random}()$ 
4:    $expected\_lc := g \cdot C$   $\triangleright C$  number of operations to be accounted for in Verify_Equality
   procedure
5:      $rand \in \mathbb{Z}^+ \leftarrow \text{Sample\_Random}()$ 
6:      $verify\_result = rand$   $\triangleright$  Initializing result of Verify_Equality
7:     Sample secret  $s \in R_q^k$  and error  $e \in R_q^k$ 
8:      $verify\_result = \text{Verify\_Equality}(s, e, lc, g)$   $\triangleright$  If Verify_Equality fails,
        $verify\_result \neq rand$ 
9:      $tmp = \text{Compute\_Public\_Key}()$   $\triangleright$  Compute  $pk$  and store in  $tmp$  array
10:    for  $j$  from 0 to  $(nb - 1)$  do  $\triangleright$  Copy  $nb$  bytes of public key from  $tmp$  to  $pk_{out}$ 
11:      if  $(expected\_lc == lc)$  and  $(expected\_lc \neq 0)$  and  $(verify\_result == rand)$  and
         $(verify\_result \neq 0)$  then
12:         $pk_{out}[j] = tmp$   $\triangleright$  Copy the public key byte if verification passes...
13:      else
14:         $pk_{out}[j] = rand()$   $\triangleright$  Copy a random byte if verification passes...
15:      end if
16:    end for
17: end procedure

```

This ensures that the correct public key is generated at the output only when the verification procedure has passed and that it has been fully executed. This comparison is done for every byte moved from tmp to pk_{out} (Line 12). Else, a random value is copied into every byte of the public key (Line 14). One can also augment with checks for a non-zero value for lc and $verify_result$ to prevent against zeroization attacks.

We argue that Verify_Nonce_Fault provides improved resistance against fault attacks in the following manner:

- (1) Zeroization of variables such as g (Line 3 in Algorithm 5) or $rand$ (Line 3) cannot pass verification in Line 11, thereby generating a random public key, which is not useful for an attacker.
- (2) Skipping the Verify_Equality procedure (Line 8) also ensures that the loop counter verification fails (Line 11), generating a random public key and thereby offering protection.
- (3) The value of lc and $verify_result$ change for every execution, and thus injection of precise faults on these variables to force successful comparison is challenging to achieve in practice (Line 11). Even if the attacker can force successful comparison, this has to be repeated for a few thousand bytes of the public key pk for Kyber KEM, which is also challenging to achieve in practice (Line 11).

Thus, we argue that it is possible to design the protection such that the implementation is not susceptible to trivial fault attacks and that the attacker requires to inject several highly synchronized faults to bypass the Verify_Nonce_Fault protection. While we choose to output a random public key upon detection of a fault, it is also possible to rerun the key-generation procedure until generation of a non-faulty public key. However, for both the options, the internal Verify_Equality procedure and Dynamic_Loop_Counter procedure remain the same.

4.1.2 Targeting NTT. Ravi et al. [64] proposed a novel fault attack targeting the NTT operation. They identified a single point of failure, that can be targeted through faults, to *zeroize all twiddle*

factors used within the NTT. When this is targeted on the NTT over secrets or errors, it can severely reduce the entropy of secret/error. This results in faulty-yet-valid LWE instances, which easily compromise the secret key. For instance, let the DUT compute NTT over the polynomial $\mathbf{x} = (x_0, x_1, \dots, x_{n-1})$. If the twiddle factors used in the NTT computation are zeroized, then the resulting faulty NTT output is $\hat{\mathbf{x}}^* = (x_0, 0, \dots, 0)$. If $\hat{\mathbf{x}}^*$ is used in subsequent computations, then the faulty polynomial $\mathbf{x}^*$ is nothing but $\mathbf{x}^* = (x_0, x_0, \dots, x_0)$. Thus, the injected fault has effectively reduced the entropy of $\mathbf{x}$ to just a single coefficient x_0 , which can be easily guessed by an attacker, given the short span of secrets and errors used in Kyber KEM.

The authors studied the assembly optimized implementation of NTT for Kyber and Dilithium on the ARM Cortex-M4 device, available in the *pqm4* library [37]. They showed that a single targeted fault using EMFI on the address pointer to the twiddle factor array can be used to effectively zeroize all the twiddle factors. In this respect, an attacker can target the NTT over the secret s or error e in the key-generation procedure (Lines 5, 6 in Algorithm 1). This results in utilization of low-entropy secrets and errors to generate a faulty public key, which can be easily solved to recover the secret key. The same faulty secret key is also used within the decapsulation procedure, as Kyber saves the secret key in the NTT domain. This, therefore, ensures correctness of Kyber KEM, even upon injection of fault in the NTT, only during key generation. We refer to this attack using the label `NTT_Twiddle_Fault` attack. The attack characteristic can be defined using the tuple: (EMFI, Observe_DUT_IO, Targeted_Fault, 1, 1).

Countermeasure: Ravi et al. [64] proposed a few detection-based countermeasures to test zeroization of the twiddle constants before utilization for the NTT operation. One can also adopt a small testing procedure to check if the twiddle factors to be used for the NTT all have a non-zero value. If one or more twiddle constants have a zero value, then the entire procedure can be aborted. As an additional protection, one can also test the entropy of the NTT output, given that the faulty NTT output has a very low entropy with a single non-zero coefficient. We refer to these detection countermeasures using the label `NTT_Twiddle_Check`.

Refer to Table 2 for a tabulation of all fault-injection attacks on the key-generation procedure of Kyber KEM.

4.2 FIA on Encapsulation

The fault attacks applicable to the key generation procedure, i.e., `Nonce_Fault` and `NTT_Twiddle_Fault` attack, are also applicable to the encapsulation procedure. The `Nonce_Fault` attack on the encapsulation procedure can be done by targeting the nonces used to sample the ephemeral secret $\mathbf{r}$ (Line 14 in Algorithm 1), whose knowledge can be used to perform message recovery. Similarly, the `NTT_Twiddle_Fault` attack can be mounted by targeting the NTT operation over $\mathbf{r}$ (Line 17), which reduces the entropy of $\mathbf{r}$ resulting in message recovery. Thus, the attacks over the key-generation procedure apply in the same manner to the encapsulation procedure of Kyber KEM.

Countermeasure: The `Verify_Nonce_Fault` and `NTT_Twiddle_Check` serve as concrete countermeasures against the aforementioned attacks on the encapsulation procedure of Kyber KEM.

Refer to Table 2 for a tabulation of all fault-injection attacks on the encapsulation procedure of Kyber KEM.

4.3 FIA on Decapsulation

With respect to FIA on the decapsulation procedure, we consider two scenarios. In case of ephemeral key setting, faulting the decapsulation procedure does not provide any information about the secret key or the message. The attacker can only inject faults to corrupt decapsulation of valid ciphertexts, which amounts to a **Denial of Service (DoS)** attack. However, in case of the

static key setting, a Communicate_DUT_IO attacker can query the DUT with chosen-ciphertexts, and the result of corresponding faulty decapsulations can potentially recover the long-term secret key. The following are different fault attacks reported on the decapsulation procedure.

4.3.1 Targeting Ciphertext Equality Check. One obvious target within the decapsulation procedure is to simply skip the final ciphertext comparison operation, whose result indicates the validity of the ciphertext (Line 21 in Algorithm 2). An attacker who can skip the equality check for his/her chosen-ciphertexts effectively reduces the security from IND-CCA security to IND-CPA security. Skipping the equality check ensures that the session key K contains critical information about the decrypted message m' , even for the attacker's chosen ciphertexts. This knowledge of the session keys for several such chosen-ciphertexts leads to recovery of the long-term secret key s .

Recently, Xagawa et al. [75] surveyed optimized software implementations of several PQC schemes on the ARM Cortex-M4 microcontroller and identified that implementations of several schemes including Kyber KEM are vulnerable to trivial skipping fault attacks. The ciphertext equality check within the optimized software implementation of Kyber KEM from the *pqm4* library [37] is done in the following manner: The pre-key $\bar{K}'$ is computed using m' and pk after decryption (Line 18 in Algorithm 2) and is stored in an array T (Line 19). If ciphertext comparison fails (invalid/malicious ciphertext), then a pseudo-random value z is written into T using a conditional move operation (Line 22). Else, the pre-key in the array T is not overwritten. Then, T is used to derive the final shared session key K (Line 24).

The vulnerability is that the decapsulation procedure writes the sensitive pre-key $\bar{K}'$ onto T (assuming successful decapsulation) before checking the validity of the ciphertext. Thus, simply skipping the subsequent conditional move operation (Line 22) for malicious ciphertexts ensures that $\bar{K}'$ is used to generate the shared session key K instead of the pseudo-random z , even for invalid ciphertexts. Xagawa et al. [75] exploited this vulnerability through simple clock glitches and could subsequently recover the secret key in a few thousand chosen-ciphertext queries, similar to the Binary_PC_Oracle_CCA attack [21, 63]. We refer to this attack using the label Skip_CT_Compare. The attack characteristic can be defined by the tuple: (Glitching, Communicate_DUT_IO, Targeted_Fault, 1, $1k-3k$).

Countermeasure (Our Proposal): We propose two levels of protection for the ciphertext comparison operation, targeted by the Skip_CT_Compare attack. Trivial skipping of the entire ciphertext comparison operation (Line 21 in Algorithm 2) can be detected through the Dynamic_Loop_Counter protection (refer to Section 4.1.1). As a second level of protection, we propose to remove the vulnerability that allows for trivial skipping attacks. We alter the conditional move operation (Line 22 in Algorithm 2) in the following manner: We ensure that the pre-key $\bar{K}'$ is written into a temporary variable *tmp* (initialized with a random value). Subsequently, the *tmp* variable containing the pre-key is copied into the array T , one byte at a time, only if both the following conditions are satisfied: (1) ciphertext comparison succeeds and (2) Dynamic_Loop_Counter verification passes. Both these checks are done for every byte that is copied from *tmp* to T (32 bytes). If either of the conditions fail, then the pseudo-random value z is copied into T . We refer to this two-stage protection using the label Protect_CT_Compare. The implementation of this countermeasure can be done in a similar manner as that of the Verify_Nonce_Fault countermeasure, and we thus refer the reader to Section 4.1.1 for more details on the implementation and effectiveness of the countermeasure.

4.3.2 Ineffective Fault Analysis. Pessl and Prokop [52] recently proposed a novel ineffective fault attack against the decapsulation procedure. It works by injecting targeted faults within the

message decoding operation during decryption (Line 28 in Algorithm 1), such that the resulting success/failure of decapsulation can be used to infer critical information about the secret key.

We briefly describe the main idea of their attack. The attacker constructs a valid ciphertext ct and submits ct for decapsulation by the DUT. Subsequently, a targeted fault is injected to skip the addition operation during decoding of a message polynomial coefficient $m'[i]$ into the message bit m'_i (refer to the code snippet of the message decoding procedure in Figure 1). The injected fault results in a flip of m'_i (decapsulation failure) only if the corresponding coefficient of the noise component $d[i] < 0$ (refer to Equation (3)). However, there is no change in m'_i when $d[i] \geq 0$ (decapsulation success).

Thus, the knowledge of whether the injected fault resulted in a decapsulation success/failure helps infer information about $d[i]$, which is linearly dependent upon the secret key s . This can be done for several valid ciphertexts to fully recover the secret key in $6.5k-13k$ queries for Kyber KEM. However, the number of queries for key recovery can be reduced to $5k-7k$ using improved post-processing techniques, as shown in Reference [33]. In essence, the attack utilizes fault injection to realize a practical **decryption failure (DF)** oracle for valid ciphertexts for key recovery. The attack was demonstrated using clock glitching on the ARM Cortex-M4 microcontroller and requires to inject a targeted skipping fault in the message decoding procedure. We refer to this attack using the label `Ineffective_FIA`. The attack characteristic can be described using the tuple: (Glitching, Communicate_DUT_IO, Targeted_Fault, 1, $5k-7k$).

Countermeasure: Since the attack specifically targets the message decoding procedure, simply shuffling the message decoding procedure, i.e., `Shuffled_Decompile`, serves as a concrete countermeasure against the attack.

4.3.3 Fault Correction Attack. Hermelink et al. [33] proposed a novel fault attack on the decapsulation procedure, which adopts a slightly different approach. The attacker constructs a valid ciphertext $ct = (u, v)$ and adds a single-bit perturbation of $\approx q/4$ to one of the coefficients of v , i.e., $v[i]$. This perturbed ciphertext $ct' = (u', v')$ is submitted to the DUT for decapsulation. Upon submitting the perturbed ciphertext, a fault is injected any time after decryption (Line 17 in Algorithm 2) and before ciphertext comparison (Line 21) to correct the single-bit perturbation in the ciphertext stored in memory. If the introduced perturbation resulted in correct decryption, then the injected fault corrects the single-bit perturbation in the ciphertext, ensuring successful decapsulation. However, if the initial perturbation resulted in a decryption failure ($d[i] < 0$), then it results in decapsulation failure, even after correcting the perturbation in the stored ciphertext through faults, since all the ciphertext coefficients of ct_R are uniformly randomized during re-encryption. This information obtained about d over $5k-7k$ such queries can recover the full secret key.

Unlike the attack of Pessl and Prokop, the attack of Hermelink et al. [33] does not have any timing constraints for fault injection, as it only needs to inject a bit-flip fault in memory anytime between the decryption and ciphertext comparison operation. However, injecting precise single bit-flip faults in memory requires detailed information about the target device as well as the implementation and an extensive profiling of the target device. The attack characteristic can be defined by the following tuple: (LFI, Communicate_DUT_IO, Targeted_Fault, 1, $5k-7k$).

More recently, Delvaux [22] improved the attack of Hermelink et al. [33] by expanding the attack surface to several operations within the decapsulation procedure while also working with a variety of more relaxed fault models such as arbitrary bit flips, set-to-0 faults, random faults, and instruction skip faults. However, attacks relying on a relaxed fault model could require about $100k$ chosen-ciphertext queries for full key recovery, depending upon the practicality of the fault model. The attack characteristic can be defined by the following tuple: (Glitching,

Table 2. Tabulation of Reported FIA and Their Characteristics for the Different Procedures of Kyber KEM

Attack	Attack Characteristic					
	Attack_Vector	DUT_IO_Access	Targeted_Or_Not	Num_Faults	Num_Executions	Countermeasure
Key Generation						
Nonce_Fault [62]	EMFI	Observe_DUT_IO	Targeted_Fault	4–8	1	Verify_Nonce_Fault
NTT_Twiddle_Fault [64]	EMFI	Observe_DUT_IO	Targeted_Fault	1	1	NTT_Twiddle_Check
Encapsulation						
Nonce_Fault [62]	EMFI	Observe_DUT_IO	Targeted_Fault	4–8	1	Verify_Nonce_Fault
NTT_Twiddle_Fault [64]	EMFI	Observe_DUT_IO	Targeted_Fault	1	1	NTT_Twiddle_Check
Decapsulation (Static Key)						
Skip_CT_Compare [75]	Glitching	Communicate_DUT_IO	Targeted_Fault	1	1k–3k	Protect_CT_Compare
Ineffective_FIA [52]	Glitching	Communicate_DUT_IO	Targeted_Fault	1	5k–7k	Shuffled_Decode
Fault_Correction [22, 33]	LFI [33]	Communicate_DUT_IO	Targeted_Fault	1	5k–7k	Message_Poly_Sanity_Check
	Glitching [22]	Communicate_DUT_IO	Targeted_Fault	1	10k–100k	

Communicate_DUT_IO, Targeted_Fault, 1, 10k–100k). We refer to the aforementioned attacks using the label Fault_Correction attack.

Countermeasure (Our proposal): We observe that the attack works with perturbed ciphertexts and observe that the corresponding coefficients of the erroneous message polynomial $\mathbf{m}'$ upon decryption do not satisfy the distribution of the message polynomial of a valid ciphertext. Thus, our proposed Message_Poly_Sanity_Check serves as a concrete detection countermeasure against the attack.

Refer to Table 2 for a tabulation of all fault-injection attacks on the decapsulation procedure of Kyber KEM in the static key setting.

5 FAULT-INJECTION ATTACKS ON DILITHIUM

In this section, we discuss fault attacks that are applicable to the Dilithium signature scheme. We utilize the same characteristics to describe FIA on Dilithium that were used to describe FIA on Kyber (refer to Section 4). We utilize the algorithm of Dilithium signature scheme in Algorithms 3–4 to explain the different attacks. We note that the secret key sk of Dilithium has multiple components: $sk = (seed_A, K, tr, s_1, s_2, t_0)$ (Line 9 in Algorithm 3). Among them, we refer to s_1 as the primary secret, since the knowledge of s_1 is sufficient to forge signatures of Dilithium for any chosen message, as shown in References [15, 60].

5.1 FIA on Key Generation

The key generation procedure of Dilithium can serve as an attractive target for fault attacks when the application utilizes self-signed certificates, where key generation is performed on the DUT. In this scenario, the following attacks are applicable to the key generation procedure:

5.1.1 Targeting Sampling of Secrets (Nonce_Fault). The polynomials of the secret s_1 and s_2 of Dilithium are sampled using the same seed $seeds$, but with different delimiters/nonces (Line 3 of Sign in Algorithm 3). Ravi et al. [62] showed that an attacker can force nonce reuse through faults to generate weak LWE instances, which can be potentially solved to recover the secret key. Dilithium utilizes rounding of the public key (Line 7), which poses an additional challenge for the attacker to recover the secret key. Nevertheless, the induced nonce reuse through faults significantly reduces the security of the public keys, as the full public key can be reconstructed by observing several valid signatures. The attack characteristic is defined using the tuple: (EMFI, Observe_DUT_IO, Targeted_Fault, 8–15, 1).

Countermeasure (Our Proposal): The Verify_Nonce_Fault countermeasure (Section 4.1.1) can serve as a concrete protection against the Nonce_Fault attack.

5.1.2 Targeting NTT (NTT_Twiddle_Fault). Ravi et al. [64] proposed to target the NTT instances through the NTT_Twiddle_Fault attack, in the key generation procedure of Kyber KEM,

to create faulty yet valid secret keys with very low entropy. While NTT is also computed over the secret key component s_1 in Dilithium, the fault attack is not applicable to the key generation procedure of Dilithium. This is because the faulty NTT transformed version of the secret s_1 is only used to generate the LWE instance, i.e., public key (Line 6 in Algorithm 3). The key-generation procedure, however, saves the original secret s_1 in the normal domain as the secret key. Thus, the signing procedure performs a fresh NTT computation over the secret s_1 while generating signatures. This violates the correctness of the generated signatures, thereby rendering the attack on the key generation procedure of Dilithium useless.

Refer to Table 3 for a tabulation of all fault-injection attacks on the key-generation procedure of Dilithium signature scheme.

5.2 FIA on Signing Procedure

The signing procedure of Dilithium remains the main target of fault injection attacks, as the signing procedure utilizes the long-term secret key sk to generate multiple signatures, given the long-lifetime of the key pairs used in signature schemes. The following attacks are applicable to the signing procedure of Dilithium:

5.2.1 Injecting Random Faults on the Secret Key. Bindel et al. [12] reported the first fault vulnerability analysis of lattice-based signature schemes such as GLP [28] and BLISS [23], based on the “Fiat-Shamir with Aborts” framework. They proposed to inject random faults to change a single or few coefficients of the secret module $s_1 \in R_q^\ell$. The attacker can subsequently utilize the knowledge of $\approx 1k-2k$ faulty signatures to obtain knowledge about the originally perturbed coefficients of s_1 one at a time to fully recover s_1 .

Along the same lines, Islam et al. [35] recently presented a novel signature correction attack, which also works by injecting random bit flips in single coefficients of the secret module s_1 , stored in memory. They utilize RowHammer as an attack vector to inject random bit flips and subsequently utilized a signature correction algorithm on the faulty signatures to recover the secret key. We henceforth refer to these attacks faulting the secret key as Randomize_Secret_Key fault attacks. The attack does not require communication with the signing DUT and can work on both the deterministic and probabilistic variants of Dilithium. The attack characteristic is defined using the tuple: (EMFI, Observe_DUT_IO, Targeted_Fault, 1, $\approx 1k-2k$).

Countermeasure: The faulty signatures generated due to injection of randomization faults are invalid with an overwhelming probability. Thus, verifying the validity of the generated signatures serves as a concrete countermeasure. The countermeasure is also effective against any future fault attacks that produce invalid signatures. We henceforth refer to this countermeasure using the label Verify_After_Sign. While this countermeasure has been proposed by several works [12, 15], its concrete implementation and performance evaluation has not been studied by prior works.

5.2.2 Generic Differential Fault Analysis (DFA). Bruinderink and Pessl [15] presented a powerful Differential Fault Attack (DFA), particularly applicable to the deterministic variant of Dilithium, whose *modus operandi* is as follows: The attacker has access to a signing oracle (Communicate_DUT_IO) and submits a signature query for a randomly chosen message m . Let the primary signature component be $z = s_1 \cdot c + y$ (Line 27 in Algorithm 3). The attacker again submits a signing query for the same message m but injects a random fault such that the corresponding faulty signature is $z' = s_1 \cdot c' + y$, which is computed with the same nonce y , but with a different challenge polynomial c' . The difference $\Delta z = z - z'$ can be used to trivially recover the entire secret module s_1 with only a single faulty signature. The authors showed that only a single random fault (using glitches) anywhere within 68% of the execution time of a single iteration of

the signing procedure can result in full key recovery, thereby demonstrating the effectiveness of their attack. Referring to the signing procedure in Algorithm 3, the random fault can be injected anywhere in lines 12 and 23–27. We henceforth refer to this attack as the Generic_DFA attack on Dilithium. Since the attack is a DFA-style attack, it can only work on the deterministic variant of Dilithium, but not on the probabilistic variant. The attack characteristic can be defined by the following tuple: (Glitching, Communicate_DUT_IO, Non_Targeted_Fault, 1, 1).

Countermeasure: Similar to the Randomize_Secret_Key attack, Generic_DFA attack also results in invalid signatures that do not pass verification. Thus, the Verify_After_Sign countermeasure serves as a strong deterrent against the attack. However, the authors of Reference [15] also showed an interesting variant of their attack, which works by injecting faults during sampling of y , that results in valid signatures. Thus, this variant of their attack can bypass the Verify_After_Sign countermeasure. However, converting the signing procedure to being probabilistic also serves as a concrete countermeasure against the attack.

5.2.3 Loop Abort Fault Attack. Espitau et al. [25] proposed a novel fault attack to directly target the nonce y in Fiat-Shamir Abort-based signature schemes such as GLP signature scheme [28] and BLISS [23]. They proposed to use faults to prematurely abort the loop, which samples the single coefficients of y (Line 22 in Algorithm 3), thereby resulting in generation of nonces with low degrees. In other words, by skipping the loop that samples individual coefficients of y , one can ensure that the remaining coefficients of y are not sampled, and there is a high chance that these unsampled coefficients have a value of 0. If so, then the faulted signature z contains several coefficients that are nothing but the unmasked coefficients of the product $s_1 \cdot c$ (Line 27 in Algorithm 3). The authors show that a single targeted fault in the sampling procedure of y can result in full key recovery. Though this attack was only demonstrated on the GLP signature scheme [28], this attack can potentially be applied to Dilithium for full key recovery. Since this attack does not involve differential analysis, it is therefore applicable to both the probabilistic and deterministic variants of Dilithium. We refer to this attack using the label Loop_Abort_Fault. Its characteristics (plural) can be defined using the tuple: (Glitching, Observe_DUT_IO, Targeted_Fault, 1, 1).

Countermeasure (Our Proposal): We propose a two-level protection mechanism, similar to that of the Verify_Nonce_Fault (Section 4.1.1), which works in the following manner: First, we utilize the Dynamic_Loop_Counter protection to keep track of the number of sampled coefficients of y . The generated signature σ is stored in a temporary variable *temp* and is copied one byte at a time to the output variable *sig* (initialized with 0) only if the the loop counter comparison succeeds, and this comparison is done for every byte copied from *temp* to *sig*. We refer to this two-level countermeasure using the label Verify_Loop_Abort. We refer to Section 4.1.1 for more details on the implementation and effectiveness of the countermeasure.

5.2.4 Skip Addition Attack. Bindel et al. [12] proposed theoretical skipping fault attacks targeting the final addition operation used to generate z (Line 27 in Algorithm 3). Skipping the addition of y with the product $(s_1 \cdot c)$ unmask the coefficients of the product $(s_1 \cdot c)$, whose knowledge can be used to recover s_1 . While this is possible by skipping the entire addition operation, Ravi et al. [60] proposed a more subtle fault attack on the deterministic variant of Dilithium, which involves skipping of the addition operation for single coefficients of z (Line 27 in Algorithm 3). An attacker can then use a DFA technique similar to Reference [15] to recover the secret module s_1 in $\approx 1k-2k$ such faulty signatures. While the attack has only been demonstrated on the deterministic variant of Dilithium, its applicability to the probabilistic variant is not clear and is yet to be studied. We refer to these attacks as the Skip_Addition fault attacks, whose characteristics (plural) can be defined using the tuple: (EMFI, Communicate_DUT_IO, Targeted_Fault, 1, $\approx 1k-2k$).

Countermeasure: The use of a Verify_Loop_Abort-like countermeasure can be used to detect skipping of any of the addition operations to generate the primary signature component z . However, the protection does not defeat attacks that skip addition of single coefficients through corruption of underlying assembly instructions [60], since they do not affect the loop counter. In this respect, Ravi et al. [60] proposed to compute the addition operation in the NTT domain, i.e., compute z as $\text{INTT}((\hat{s}_1 \circ \hat{c}) + \hat{y})$, i.e., alternative to computation of z in Line 27 in Algorithm 3. Thus, skipping fault in at least one coefficient of z uniformly propagates the fault to all coefficients through the subsequent INTT operation. This results in an invalid signature that is rejected by the conditional check on $\|z\|_\infty$ with a very high probability (Line 29 in Algorithm 3). We propose to utilize the Dynamic_Loop_Counter protection along with addition in the NTT domain, which is referred to as the Verify_Add countermeasure.

5.2.5 Targeting NTT (NTT_Twiddle_Fault). Ravi et al. [64] proposed to inject faults to zeroize the twiddle constants of specific NTT instances in the signing procedure to generate faulty signatures, which compromise the secret key. They proposed two variants of attacks. The first attack variant works on the deterministic variant of Dilithium in the following manner: The attacker obtains a valid signature $\sigma = (z, h, c)$ of a message μ , with the constraint that $c[0] = 0$ (first coefficient of c). Let $z = s_1 \cdot c + y$. Subsequently, the attacker submits a signing query for the same message but now injects a fault in the NTT instance of c (Line 26 in Algorithm 3). This effectively zeroizes the entire NTT output of c , i.e., $\hat{c}$ (Line 26). Thus, the generated faulty signature is nothing but $z^* = y$. The difference of z and z^* can be used to trivially recover s_1 , similar to the Generic_DFA attack. This attack only works on the deterministic variant and cannot work on the probabilistic variant, since it is a DFA-style attack. The attack characteristic is denoted using the tuple: (EMFI, Communicate_DUT_IO, Targeted_Fault, 1, 1).

The authors also proposed a non-DFA-style variant of the attack that can work on the probabilistic variant, but when z is computed as $\text{INTT}((\hat{s}_1 \circ \hat{c}) + \hat{y})$, similar to the Verify_Add countermeasure. They propose to fault the NTT over y (Line 22), which zeroizes all except the first coefficient of all the polynomials of y . Thus, the resulting faulty signature component z^* is nothing but $s_1 \cdot c$, except for the first coefficient of every polynomial of z^* . The complete secret key s_1 can be recovered in a single such targeted fault. Moreover, the attacker does not require to communicate with the signing DUT for the attack. Thus, the attack characteristic is denoted using the tuple: (EMFI, Observe_DUT_IO, Targeted_Fault, 1, 1).

Countermeasure: The NTT_Twiddle_Check countermeasure that verifies the sanity of the twiddle factors can be used as a concrete countermeasure against the attack (refer to Section 4.1.2).

Refer to Table 3 for a tabulation of all fault-injection attacks on the signing procedure of Dilithium signature scheme.

5.3 FIA on Verification Procedure

While the aforementioned attacks target the signing procedure, the verification procedure could also serve as a good target for fault injection attacks. One of the main motivations being forceful acceptance of invalid signatures through faults for any message of the attacker's choice.

5.3.1 Targeting NTT (NTT_Twiddle_Fault). Ravi et al. [64] proposed a fault attack that zeroizes the twiddle constants of the NTT over the challenge polynomial c in the verification procedure (Line 3 in Algorithm 4). They also proposed a forgery algorithm, which can be used to enforce successful verification for any message of the attacker's choice if an attacker can achieve the aforementioned fault. We utilize the following tuple to define the attack characteristic: (EMFI, Communicate_DUT_IO, Targeted_Fault, 1, 1).

Table 3. Tabulation of Reported FIA and Their Characteristics for the Different Procedures of Dilithium Signature Scheme

Attack	Attack Characteristic					
	Attack_Vector	DUT_IO_Access	Targeted_Or_Not	Num_Faults	Num_Executions	Countermeasure
Key Generation						
Nonce_Fault [62]	EMFI	Observe_DUT_IO	Targeted_Fault	1-10	1	Verify_Nonce_Fault
NTT_Twiddle_Fault [64]	EMFI	Observe_DUT_IO	Targeted_Fault	1	1	NTT_Twiddle_Check
Signing						
Randomize_Secret_Key [12, 35]	EMFI	Observe_DUT_IO	Targeted_Fault	1	1k-2k	Verify_After_Sign
Generic_DFA [15]	Glitching	Communicate_DUT_IO	Non_Targeted_Fault	1	1	Verify_After_Sign
Loop_Abort_Fault [25]	Glitching	Observe_DUT_IO	Targeted_Fault	1	1	Verify_Loop_Abort
Skip_Addition [12, 60]	EMFI	Communicate_DUT_IO	Targeted_Fault	1	1k-2k	Verify_Add
NTT_Twiddle_Fault [64]	EMFI	Communicate_DUT_IO	Targeted_Fault	1	1	NTT_Twiddle_Check
	EMFI	Observe_DUT_IO	Targeted_Fault	1	1	NTT_Twiddle_Check
Verification						
NTT_Twiddle_Fault [64]	EMFI	Communicate_DUT_IO	Targeted_Fault	1	1	NTT_Twiddle_Check
Skip_C_Compare [75]	Glitching	Communicate_DUT_IO	Targeted_Fault	1	1	Dynamic_Loop_Counter

Countermeasure: The NTT_Twiddle_Check countermeasure that verifies the sanity of the twiddle factors can be used as a concrete countermeasure against the attack (refer to Section 4.1.2).

5.3.2 Skipping Equality Check. One of the obvious targets for fault injection is to simply skip the final comparison operation that decides the validity of the received signatures. In particular, bypassing the comparison of the received challenge polynomial c with the recomputed challenge polynomial $\bar{c}$ (Line 6 in Algorithm 4) ensures successful signature verification. This attack is very similar to the Skip_CT_Compare attack on KEMs, targeting the ciphertext comparison operation in the decapsulation procedure. While the attack has not been practically demonstrated, it is important to fortify the equality check in the verification procedure to prevent trivial skipping attacks. We refer to this attack using the label Skip_C_Compare.

Countermeasure (Our Proposal): We propose to simply utilize a Dynamic_Loop_Counter countermeasure to keep track of the number of compared coefficients of the challenge polynomial. This loop counter information along with the result of the comparison operation can be used to protect against trivial skipping attacks. One can also adopt redundancy of varying degrees to further fortify the verification procedure. We agree that this is only an implementation-level countermeasure and can therefore be circumvented by a more powerful attacker. However, these countermeasures do significantly increase the ability of an attacker to mount a successful attack.

Refer to Table 3 for a tabulation of all fault-injection attacks on the verification procedure of Dilithium signature scheme.

6 SIDE-CHANNEL ATTACKS ON DILITHIUM

In this section, we discuss side-channel attacks that are applicable to the Dilithium signature scheme. We only consider side-channel attacks on the key-generation and signing procedure as they manipulate the secret key, while verification procedure that manipulates public information is not relevant for side-channel attacks. We utilize the same characteristics to describe SCA on Dilithium that were used to describe SCA on Kyber (refer to Section 3.1.4). We utilize the algorithm of Dilithium signature scheme in Algorithms 3-4 to explain the different attacks.

6.1 SCA on Key Generation

6.1.1 SASCA. The key generation procedure of Dilithium is susceptible to SASCA-based attacks.

Targeting NTT (SASCA_NTT): Leakage from the NTT instance over the primary secret key component s_1 (Line 5 of KeyGen in Algorithm 3) can be used to recover s_1 in a single trace.

Targeting KECCAK (SASCA_KECCAK): KECCAK is used as a PRNG within the key-generation procedure to sample the secret s_1 (Line 3 in Algorithm 3) using $seeds$, thus SASCA on this KECCAK instance can be potentially used to recover $seeds$, which can be used to reconstruct the secret key s_1 .

The SASCA_NTT and SASCA_KECCAK attacks can be defined using the tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Countermeasure: Shuffling the sensitive NTT as well as the KECCAK operations provides concrete protection against attacks relying on SASCA.

6.1.2 Simple Template Attacks. Han et al. [31] targeted the NTT instance over s_1 using a simple template attack, which could recover the complete secret polynomial s_1 in a single trace. They showed that an attacker can target leakage from the product of the secret coefficients with the twiddle factors in the first round of the NTT, i.e., $prod \in \mathbb{Z}_q = s[i] \cdot \omega^j$, where $s[i]$ is a secret coefficient and ω^j is a twiddle factor. They show that leakage of the result $prod$ can be used to uniquely distinguish every candidate of $s[i]$ through simple template attacks. Moreover, the attack is aided by the fact that there are only five possible candidates for coefficients of the secret s_1 . Han et al. [31] targeted the reference implementation of Dilithium through the power side-channel on the ARM Cortex-M4 microcontroller to recover the entire secret s_1 in a single trace. We refer to this attack as the Simple_NTT_Template attack. Similar to SASCA on the NTT, the attack can be defined using the tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Countermeasure: Unlike SASCA on NTT, which relies on leakage from intermediate variables throughout the NTT operation, this attack only relies on leakage from a single intermediate variable for key recovery. Thus, the leakage exploited is more fine-grained and is more prone to noise (horizontal/vertical) compared to SASCA-type attacks. Nevertheless, the shuffling and masking countermeasures proposed for the NTT serve as a concrete countermeasure against this attack (Shuffled_Masked_NTT [61]).

Refer to Table 4 for a tabulation of all side-channel attacks on the key-generation procedure of Dilithium signature scheme.

6.2 SCA on Signing Procedure

6.2.1 SASCA. The signing procedure of Dilithium is susceptible to SASCA-based attacks.

Targeting NTT (SASCA_NTT): Similar to the key-generation procedure, the signing procedure also computes NTT of the primary secret s_1 (Line 15 of Sign Algorithm 3), which can be targeted using SASCA for single trace key recovery. Similarly, NTT instance over the ephemeral nonce y (Line 23) can also be targeted, whose knowledge can be used to recover the primary secret s_1 .

Targeting KECCAK (SASCA_KECCAK): KECCAK is used as a PRNG within the signing procedure to sample the ephemeral nonce y from a small seed ρ (Line 22), which is vulnerable to single-trace SASCA_KECCAK attacks.

The SASCA_NTT and SASCA_KECCAK attacks can work on both the deterministic and probabilistic variants of Dilithium and can be defined using the tuple: (Observe_DUT_IO, Profiled_With_Clone, 1, High_SNR).

Countermeasure: Shuffling the NTT as well as the KECCAK operation provides concrete protection against attacks relying on SASCA.

6.2.2 Targeting the Nonce y . Recently, Marzougui et al. [42] demonstrated a profiled attack targeting the sampling of the ephemeral nonce y (Line 22 in Algorithm 22). They proposed to profile the leakage of coefficients of y using a machine learning classifier to differentiate between a given

coefficient $y[i] = 0$ and $y \neq 0$. Templates for the coefficients of y can be built using leakage from a clone device. During the attack phase, a single trace is obtained and the attacker attempts to exploit leakage from single coefficients of y to identify zero coefficients of y . If a given coefficient $y[i] = 0$, then $z[i] = s_1 \cdot c[i]$, and if an attacker can identify $\ell \cdot n$ such coefficients, then he can fully recover the secret key using simple Gaussian elimination. Marzougui et al. [42] performed their attack on ARM Cortex-M4 microcontroller through the power side-channel and were able to recover the full key in $\approx 750k$ signatures. A very high number of signatures is required to identify coefficients of y that have a very small value close to 0, while they are uniformly distributed in the range $[0, 2^{19}]$ for recommended parameters of Dilithium. The attack also exploits fine leakages from manipulation of single coefficients and therefore requires a relatively high SNR. We refer to this attack as the Zero_Nonce_Detect attack, which can be described using the tuple: (Observe_DUT_IO, Profiled_With_Clone, $\approx 750k$, High_SNR).

Countermeasure: Masking the nonce y in the signing procedure serves as an effective countermeasure against the Zero_Nonce_Detect attack, as detecting the exact value of a coefficient using leakage from multiple shares in a single trace is not very trivial, or at the least exponentially increases the number of traces with increasing masking order. There have been several proposals for masking Dilithium against side-channel attacks [6, 43]. We refer to this countermeasure using the label Masking.

6.2.3 Correlation Power Analysis (CPA). The first CPA-style attack was proposed by Ravi et al. [59], who demonstrated a single-trace horizontal-style DPA attack targeting the operation $s_1 \cdot c$ (Line 27 in Algorithm 3), implemented using the school-book polynomial multiplier. However, they only demonstrated a simulated attack assuming idealized leakage models and, to some extent, evaluated the effect of leakage noise. Moreover, NTT is the actual polynomial multiplication algorithm used in Dilithium, and thus this attack is no more applicable to the latest implementations of Dilithium.

More recently, Chen et al. [16] demonstrated a non-profiled CPA attack targeting the point-wise multiplication of $\hat{c}$ and $\hat{s}_1$ in the NTT domain. They were able to recover the secret key in only 200 power traces using leakage from the ARM Cortex-M4 microcontroller. We refer to these attacks using the label NTT_Leakage_CPA. Since these attacks work over multiple traces, they can still work with low SNR. The attack characteristic can be described using the following tuple: (Observe_DUT_IO, Non_Profiled, ≈ 200 , Low_SNR). More recently, Steffen et al. [69] extended the CPA attack to also target the same point-wise multiplication operation in a hardware implementation on the Artix-7 FPGA, where they required about 66k traces for full key recovery, which is ≈ 300 times higher compared to targeting a software implementation on the ARM Cortex-M4 microcontroller. The NTT_Leakage_CPA attack can work on both the deterministic and probabilistic variants of Dilithium.

Countermeasure: Masking the signing procedure serves as a strong countermeasure against the aforementioned CPA-style attacks.

Refer to Table 4 for a tabulation of all side-channel attacks on the signing procedure of Dilithium signature scheme.

7 EXPERIMENTAL EVALUATION

In this section, we perform a practical performance evaluation of the presented countermeasures when integrated into optimized software implementations of Kyber and Dilithium, running on the following two embedded platforms: (1) STM32F4 microcontroller based on the ARM Cortex-M4 processor; (2) Raspberry Pi 3 Model B Plus based on the ARM Cortex-A53 processor. While

Table 4. Tabulation of Reported SCA and Their Characteristics for the Different Procedures of Dilithium Signature Scheme

Attack	Attack Characteristic					
	Attack_Vector	DUT_IO_Access	Profile_Requirement	No_Traces	SNR	Countermeasure
Key Generation						
SASCA_NTT [51, 53]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Masked_NTT
SASCA_KECCAK [36]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_KECCAK
Simple_NTT_Template [31]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Masked_NTT
Sign						
SASCA_NTT [51, 53]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_Masked_NTT
SASCA_KECCAK [36]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	1	High_SNR	Shuffled_KECCAK
Zero_Nonce_Detect [42]	Power/EM	Observe_DUT_IO	Profiled_With_Clone	≈ 750k	High_SNR	Masking
NTT_Leakage_CPA [16, 69]	Power/EM	Observe_DUT_IO	Non_Profiled	≈ 200	Low_SNR	Masking

the STM32F4 microcontroller is a representative for an low-power embedded microcontroller, the Raspberry Pi 3 device is a representative of a more sophisticated handheld gadget.

7.1 Target Platform and Implementation Details

7.1.1 ARM Cortex-M4-based Platform. Our target platform for the ARM Cortex-M4 processor is the STM32F4DISCOVERY board (DUT) housing the STM32F407 microcontroller, and the clock frequency is 24 MHz. Our countermeasures were implemented on the M4-optimized implementations of Kyber and Dilithium available in the public *pqm4* library [37], a benchmarking framework for PQC schemes on the ARM Cortex-M4 microcontroller. The M4-optimized implementation of Kyber is based on the memory-efficient high-speed implementation proposed by Botros, Kannwischer, and Schwabe in Reference [14]. The M4-optimized implementation is based on compact Dilithium optimizations reported by Greconici, Kannwischer, and Sprenkels in Reference [27].¹ Their work builds upon the early evaluation optimization by Ravi et al. [57] and additionally proposes faster assembly implementations of NTT for the Cortex-M4. All implementations were compiled with the arm-none-eabi-gcc-7.3.1 compiler using compiler flags -O3 -mthumb -mcpu=cortex-m4 -mfloat-abi=hard -mfpu=fpv4-sp-d16. We have implemented the countermeasures on both Kyber and Dilithium such that the required countermeasures can be independently turned on/off based on the designer's security requirements.

7.1.2 ARM Cortex-A53-based Platform. Our target platform for the ARM Cortex-A53 processor is the Raspberry Pi 3 Model B Plus Rev 1.3, running Debian GNU/Linux 11 at 1.4 GHz. We perform evaluation of countermeasures on the NEON-optimized implementations of Kyber and Dilithium available in the *liboqs* library [68], an open source C library for quantum-safe cryptographic algorithms. The NEON-optimized implementations of Kyber and Dilithium are based on the work of Becker et al. [9]. All implementations were compiled with the aarch64-none-linux-gnu-10.3-2021.07 compiler at the highest optimization level -O3. We have implemented the countermeasures on both Kyber and Dilithium such that the required countermeasures can be independently turned on/off based on the designer's security requirements.

7.2 Experimental Results for Kyber KEM

7.2.1 ARM Cortex-M4-based Platform. Considering the different SCA and FIA mounted on Kyber, we implement the following countermeasures within the implementation of Kyber KEM on the ARM Cortex-M4 microcontroller:

¹Our analysis and experiments were carried out on the implementations of Kyber and Dilithium corresponding to the commit hash 2691b4915b76db8b765ba89e4e09adc6b999763f and were available in the *pqm4* library until January 31, 2022.

Table 5. Performance Comparison of the Shuffled_Masked_NTT Countermeasures for Kyber KEM, Compared to the Optimized Unprotected Implementations on the ARM Cortex-M4 Device

Scheme	Clock Cycles ($\times 10^3$)								
	KeyGen			Encaps			Decaps		
	Unprot.	Prot.	Ovh. (%)	Unprot.	Prot.	Ovh. (%)	Unprot.	Prot.	Ovh. (%)
Coarse_Shuffled_NTT									
Kyber512	463	786	69	556	971	74	518	1,021	96
Kyber768	762	1,245	63	909	1,486	63	853	1,512	77
Kyber1024	1,207	1,854	53	1,386	2,125	53	1,312	2,133	62
Generic_2_Masked_NTT									
Kyber512	463	732	57	556	899	61	518	937	80
Kyber768	761	1,163	52	909	1,387	52	853	1,406	64
Kyber1024	1,207	1,744	44	1,386	1,998	44	1,312	1,999	52

Numbers were obtained on the STM32F407VG microcontroller mounted on the STM32F407DISCOVERY board, running at 24 MHz. Numbers are provided in terms of $\times 10^3$ clock cycles. Ovh. denotes overhead in percentage.

- (1) Shuffled_Masked_NTT (KeyGen, Encaps, Decaps)
- (2) Verify_Nonce_Fault (KeyGen, Encaps)
- (3) CT_Sanity_Check (Decaps)
- (4) Message_Poly_Sanity_Check (Decaps)
- (5) Protect_CT_Compare (Decaps)
- (6) Shuffled_Encode and Shuffled_Decode (Encaps, Decaps)

While we have also discussed about dedicated countermeasures such as Shuffled_KECCAK and NTT_Twiddle_Check in the article, we have not implemented them in this work. Nevertheless, the aforementioned dedicated countermeasures, either separate or combined, are not meant to serve as standalone countermeasures for Kyber but can be implemented on top of masking countermeasures for concrete protection against both SCA and FIA [13, 32].

Refer to Table 5 for the performance overheads due to the Shuffled_Masked_NTT countermeasures against the NTT_Leakage attacks on Kyber KEM, running on the ARM Cortex-M4 microcontroller. While we have implemented all the shuffling (3) and masking (4) countermeasures proposed in Reference [61], for brevity, we only report numbers for the countermeasures referred to as Coarse_Shuffled_NTT and Generic_2_Masked_NTT (refer to Reference [61] for the terminology used for the different Shuffled_Masked_NTT countermeasures).

On the ARM Cortex-M4 device, we observe a performance impact in the range of 52%–69% for key generation, 44%–74% for the encapsulation, and 52%–96% for the decapsulation procedure, across all parameters of Kyber KEM. Please note that the unprotected implementation utilizes assembly optimized NTT, while the protected implementation utilizes protected NTTs that are implemented in C. Thus, we argue that it is possible to obtain significantly improved overheads, provided that the protected NTT/INTTs are implemented in assembly. We leave the optimized implementation of these countermeasures in assembly as future work.

Refer to Table 6 for the performance overheads due to the Verify_Nonce_Fault countermeasure on the key-generation procedure, and CT_Sanity_Check, Message_Poly_Sanity_Check, Shuffle_Encode and Shuffled_Decode countermeasures for the decapsulation procedure for Kyber KEM, implemented on the ARM Cortex-M4 device. These countermeasures impose very reasonable overheads in the range of 10%–11%, 15%–34%, 12%–30%, and 4%–5% for the different parameter sets of Kyber KEM. Thus, we can see that these dedicated countermeasures can be implemented in a cost-effective manner for Kyber KEM.

Table 6. Performance Comparison of the Custom SCA-FIA Countermeasures for Kyber KEM, Compared to the Optimized Unprotected Implementation on the ARM Cortex-M4 Device

Scheme	Clock Cycles ($\times 10^3$)		Ovh. (%)
	Unprot.	Prot.	
Verify_Nonce_Fault (KeyGen)			
Kyber512	463	516	11
Kyber768	762	848	11
Kyber1024	1,207	1,337	10
CT_Sanity_Check (Decaps)			
Kyber512	518	698,157	34
Kyber768	853	1,040,291	21
Kyber1024	1,312	1,520,229	15
Message_Poly_Sanity_Check (Decaps)			
Kyber512	518	679	30
Kyber768	853	1,014	18
Kyber1024	1,312	1,473	12
Protect_CT_Compare (Decaps)			
Kyber512	518	549	5
Kyber768	853	894	4
Kyber1024	1,312	1,372	4
Shuffle_Encode_Decode (Decaps)			
Kyber512	518	586	13
Kyber768	853	878	2
Kyber1024	1,312	1,337	2

Numbers were obtained on the STM32F407VG microcontroller mounted on the STM32F407DISCOVERY board, running at 24 MHz. Numbers are provided in terms of $\times 10^3$ clock cycles. Ovh. denotes overhead in percentage.

7.2.2 ARM Cortex-A53-based Platform. Considering the different SCA and FIA mounted on Kyber, we implement the following countermeasures within the implementation of Kyber KEM on the ARM Cortex-A53 processor:

- (1) Shuffled_Masked_NTT (KeyGen, Encaps, Decaps)
- (2) CT_Sanity_Check (Decaps)
- (3) Message_Poly_Sanity_Check (Decaps)
- (4) Protect_CT_Compare (Decaps)
- (5) Shuffled_Encode and Shuffled_Encode (Encaps, Decaps)

Refer to Table 7 for the performance overheads due to the Shuffled_Masked_NTT countermeasures against the NTT_Leakage attacks on Kyber KEM, running on the ARM Cortex-A53 processor. While we have implemented all the shuffling (3) and masking (4) countermeasures proposed in Reference [61], for brevity, we only report numbers for the countermeasures referred to as Coarse_Shuffled_NTT and Generic_2_Masked_NTT.

Table 7. Performance Comparison of the Shuffled_Masked_NTT Countermeasures for Kyber KEM, Compared to the Optimized Unprotected Implementations on the ARM Cortex-A53 Device

Scheme	Clock Cycles ($\times 10^3$)								
	KeyGen			Encaps			Decaps		
	Unprot.	Prot.	Ovh. (%)	Unprot.	Prot.	Ovh. (%)	Unprot.	Prot.	Ovh. (%)
Coarse_Shuffled_NTT									
Kyber512	108.3	345.7	219	128.7	461.0	258.2	114.5	518.6	352.7
Kyber768	162.5	519.3	219.5	193.1	658.8	241	177.4	714.7	302.9
Kyber1024	244.3	719.1	194.3	285.4	880.1	208.4	267.6	934.9	249.3
Generic_2_Masked_NTT									
Kyber512	108.3	308.0	184.2	128.7	429.9	234	114.5	484.2	322.8
Kyber768	162.5	462.1	184.4	193.1	615.6	218.7	177.4	667.8	276.4
Kyber1024	244.3	643.2	163.2	285.4	825.1	189.1	267.6	874.4	226.7

Numbers were obtained on the Raspberry Pi 3 Model B Plus Rev 1.3 based on the ARM Cortex-A53 processor, running Debian GNU/Linux 11 at 1.4 GHz. Numbers are provided in terms of $\times 10^3$ clock cycles. Ovh. denotes overhead in percentage.

On the ARM Cortex-A53 device, we observe a high performance impact between 163%–219% for key generation, 189%–258% for encapsulation, and 226%–352% for decapsulation. We note that the optimized NTT implementations on the target devices are implemented in pure assembly (NEON-optimized), but the countermeasures are implemented over the C-based NTT/INTT implementations, resulting in high performance overheads on the ARM Cortex-A53 device. Thus, we argue that it is possible to obtain significantly improved overheads, provided that the protected NTT/INTTs are implemented in assembly.

It is also important to note that the NTT_Leakage attacks have only been demonstrated on the ARM Cortex-M4 device, in a profiled setting, assuming attacker's in-depth knowledge of the target device as well as implementation. Thus, these attacks are not trivial to be mounted on more complex platforms such as the ARM Cortex-A53 device, which run an embedded Linux distribution at a much higher frequency (1.4 GHz), compared to the ARM Cortex-M4 (168 MHz). Thus, in a practical setting, Shuffled_Masked_NTT countermeasures for the ARM Cortex-A53 device might not be necessary and could serve as overkill.

Refer to Table 8 for the performance overheads due to the CT_Sanity_Check, Message_Poly_Sanity_Check and Shuffled_Encode_Decode countermeasures for Kyber KEM, implemented on the ARM Cortex-A53 device.² These countermeasures impose very reasonable overheads in the range of 3%–4%, $\approx 0\%$, 7%–18%, and 19%–39% for the different parameter sets of Kyber KEM.

7.3 Experimental Results for Dilithium

7.3.1 ARM Cortex-M4-based Platform. Considering the different SCA and FIA mounted on Dilithium, we implement the following countermeasures within the implementation of Dilithium signature scheme on both the ARM Cortex-M4 and ARM Cortex-A53 microcontrollers:

- (1) Shuffled_Masked_NTT (KeyGen, Sign)
- (2) Verify_After_Sign (Sign)
- (3) Verify_Loop_Abort (Sign)

²We do not report results for the Verify_Nonce_Fault countermeasure on the ARM Cortex-A53 device in the article, but we note that the countermeasures implemented on the ARM Cortex-M4 device in C language can be easily ported to the ARM Cortex-A53 device.

Table 8. Performance Comparison of the Custom SCA-FIA Countermeasures for Kyber KEM, Compared to the Optimized Unprotected Implementations on the ARM Cortex-A53 Device

Scheme	Clock Cycles ($\times 10^3$)		
	Decaps		
	Unprot.	Prot.	Ovh. (%)
CT_Sanity_Check			
Kyber512	114.5	120.0	4.8
Kyber768	177.4	189.0	6.6
Kyber1024	267.6	277.7	3.8
Message_Poly_Sanity_Check			
Kyber512	114.5	114.1	-0.3
Kyber768	177.4	178.8	0.8
Kyber1024	267.6	268.0	0.2
Protect_CT_Compare			
Kyber512	114.5	135.3	18.1
Kyber768	177.4	200.9	13.3
Kyber1024	267.6	288.3	7.7
Shuffle_Encode_Decode			
Kyber512	114.5	159.7	39.5
Kyber768	177.4	227.5	28.2
Kyber1024	267.6	320.8	19.9

Numbers were obtained on the Raspberry Pi 3 Model B Plus Rev 1.3 based on the ARM Cortex-A53 processor, running Debian GNU/Linux 11 at 1.4 GHz. Numbers are provided in terms of $\times 10^3$ clock cycles. Ovh. denotes overhead in percentage.

(4) Verify_Add (Sign)

(5) Protect_Verify_Compare (Verify)

While we have also discussed dedicated countermeasures such as Shuffled_KECCAK and NTT_Twiddle_Check and Verify_Nonce_Fault countermeasures in the article, we have not implemented them for Dilithium in this work. Nevertheless, the aforementioned dedicated countermeasures, either separate or combined, are not meant to serve as standalone countermeasures for Dilithium, but can be implemented on top of masking countermeasures for concrete protection against both SCA and FIA [6, 43].

Refer to Table 9 for the performance overheads due to the Shuffled_Masked_NTT countermeasures against the NTT_Leakage attacks on Dilithium implemented on the ARM Cortex-M4 microcontroller. While we have implemented all the shuffling (3) and masking (4) countermeasures proposed in Reference [61], for brevity, we only report numbers for the countermeasures referred to as Coarse_Shuffled_NTT and Generic_2_Masked_NTT.

Table 9. Performance Comparison of the Shuffled_Masked_NTT Countermeasures for Dilithium, Compared to the Optimized Unprotected Implementation on the ARM Cortex-M4 Device

Scheme	Clock Cycles ($\times 10^6$)					
	KeyGen			Sign		
	Unprot.	Prot.	Ovh. (%)	Unprot.	Prot.	Ovh. (%)
Coarse_Shuffled_NTT						
Dilithium2	1.6	2.1	32	4.1	9.2	124
Dilithium3	2.8	3.5	24	6.6	15.3	132
Generic_2_Masked_NTT						
Dilithium2	1.6	2.0	30	4.1	8.9	116
Dilithium3	2.8	3.5	22	6.6	14.6	121

Numbers were obtained on the STM32F407VG microcontroller mounted on the STM32F407DISCOVERY board, running at 24 MHz. Numbers are provided in terms of $\times 10^6$ clock cycles. Ovh. denotes overhead in percentage.

On the ARM Cortex-M4 device, we observe a performance impact in the range of 22%–32% for key-generation and 116%–132% for the signing procedure. Please note that the unprotected implementation utilizes assembly-optimized NTT, while the protected implementation utilizes protected NTTs that are implemented in C. The overhead on the signing procedure is much more pronounced, since the majority of its computation time is consumed by polynomial multiplication operation. Moreover, the iterative nature of the signing procedure further increases the impact of our unoptimized protected NTT implementations. Thus, we argue that it is possible to obtain significantly improved overheads, provided that the protected NTT/INTTs are implemented in assembly. We leave the optimized implementation of these countermeasures in assembly as future work.

Refer to Table 10 for the performance overheads due to the Verify_After_Sign, Verify_Loop_Abort and Verify_Add countermeasures for the signing procedure and Protect_Verify_Compare countermeasure for the verification procedure of Dilithium, implemented on the ARM Cortex-M4 microcontroller. On the ARM Cortex-M4 device, these countermeasures impose very reasonable overheads in the range of 8%–12%, 11%–13%, and 1%–2% and 0.1%–0.05% for the different parameter sets of Dilithium.

7.3.2 ARM Cortex-A53-based Platform. Considering the different SCA and FIA mounted on Dilithium, we implement the following countermeasures within the implementation of Dilithium signature scheme on both the ARM Cortex-M4 and ARM Cortex-A53 microcontrollers:

- (1) Shuffled_Masked_NTT (KeyGen, Sign)
- (2) Verify_After_Sign (Sign)
- (3) Protect_Verify_Compare (Verify)

Refer to Table 11 for the performance overheads due to the Shuffled_Masked_NTT countermeasures against the NTT_Leakage attacks on Dilithium, implemented on the ARM Cortex-A53 device. While we have implemented all the shuffling (3) and masking (4) countermeasures proposed in Reference [61], for brevity, we only report numbers for the countermeasures referred to as Coarse_Shuffled_NTT and Generic_2_Masked_NTT.

On the ARM Cortex-A53 device, we observe a higher performance impact between 77%–109% for key generation, 438%–384% for the signing procedure, compared to the ARM Cortex-M4 device. We note that the optimized NTT implementations on the target devices are implemented in pure

Table 10. Performance Comparison of the Different SCA-FIA Countermeasures for Dilithium and the Overheads They Incur on Optimized Implementations on the ARM Cortex-M4 Device

Scheme	Clock Cycles ($\times 10^6$)		
	Unprot.	Prot.	Ovh. (%)
Verify_After_Sign (Sign)			
Dilithium2	4.1	4.6	12
Dilithium3	6.6	7.2	8
Verify_Loop_Abort (Sign)			
Dilithium2	4.1	4.7	13
Dilithium2	6.6	7.3	11
Verify_Add (Sign)			
Dilithium2	4.1	4.3	2
Dilithium2	6.6	6.7	1
Protect_Verify_Compare (Verify)			
Dilithium2	1.5	1.5	0.1
Dilithium3	2.6	2.7	0.05

Numbers were obtained on the STM32F407VG microcontroller mounted on the STM32F407DISCOVERY board, running at 24 MHz. Numbers are provided in terms of $\times 10^6$ clock cycles. Ovh. denotes overhead in percentage.

Table 11. Performance Comparison of the Shuffled_Masked_NTT Countermeasures for Dilithium, Compared to the Optimized Unprotected Implementation on the ARM Cortex-A53 Device

Scheme	Clock Cycles ($\times 10^3$)					
	KeyGen			Sign		
	Unprot.	Prot.	Ovh. (%)	Unprot.	Prot.	Ovh. (%)
Coarse_Shuffled_NTT						
Dilithium2	434.1	908.5	109.3	1,329.6	5,956.4	348
Dilithium3	745.1	1,423.2	91	1,987.5	9,194.8	362.6
Dilithium5	1,205.2	2,157.4	79	2,527.8	11,099.1	339.1
Generic_2_Masked_NTT						
Dilithium2	434.0	894.9	106.2	1,329.6	5,993.8	350.8
Dilithium3	745.1	1,415.5	90	1,987.5	9,628.5	384.5
Dilithium5	1,205.2	2,138.9	77.5	2,527.8	11,331.9	348.3

Numbers were obtained on the Raspberry Pi 3 Model B Plus Rev 1.3 based on the ARM Cortex-A53 processor, running Debian GNU/Linux 11 at 1.4 GHz. Numbers are provided in terms of $\times 10^3$ clock cycles. Ovh. denotes overhead in percentage.

assembly (M4-optimized and NEON-optimized), but the countermeasures are implemented over the C-based NTT/INTT implementations, resulting in high performance overheads, particularly on the ARM Cortex-A53 device. Thus, we argue that it is possible to obtain significantly improved overheads, provided that the protected NTT/INTTs are implemented in assembly. Refer to Table 12

Table 12. Performance Comparison of the Different SCA-FIA Countermeasures for Dilithium and the Overheads They Incur on Optimized Implementations on the ARM Cortex-A53 Device

Scheme	Clock Cycles ($\times 10^3$)		
	Unprot.	Prot.	Ovh. (%)
Verify_After_Sign (Sign)			
Dilithium2	1,329.6	1,537.0	15.6
Dilithium3	1,987.5	2,413.1	21.4
Dilithium5	2,527.8	3,155.1	24.8
Protect_Verify_Compare (Verify)			
Dilithium2	438.6	452.8	3.2
Dilithium3	699.5	740.8	5.9
Dilithium5	1,188.3	1,259.4	6

Numbers were obtained on the Raspberry Pi 3 Model B Plus Rev 1.3 based on the ARM Cortex-A53 processor, running Debian GNU/Linux 11 at 1.4 GHz. Numbers are provided in terms of $\times 10^3$ clock cycles. Ovh. denotes overhead in percentage.

for the performance overheads due to the Verify_After_Sign and Protect_Verify_Compare countermeasures for Dilithium, implemented on the ARM Cortex-A53 device. On the ARM Cortex-A53 device, these countermeasures impose very reasonable overheads in the range of 15%–24%, 32%–33%, and 3%–6% for Dilithium.³

Thus, we can observe that the presented countermeasures can be implemented in a cost-effective manner for the Dilithium signature scheme on the ARM Cortex-M4 and ARM Cortex-A53 device. It is important to note that these countermeasures are only implemented in C language, while assembly-optimized implementations can result in much smaller overheads.

8 CONCLUSION

In this work, we present a systematic study of **Side-Channel Attacks (SCA)** and **Fault Injection Attacks (FIA)** on structured lattice-based schemes, with main focus on Kyber and Dilithium, and also discuss appropriate countermeasures for each of the different attacks. Among the several countermeasures discussed in this work, we present novel countermeasures that offer simultaneous protection against several SCA- and FIA-based chosen-ciphertext attacks for Kyber KEM. Finally, we implement all the presented countermeasures for Kyber and Dilithium within two well-known public software libraries for PQC: (1) *pqm4* library for the ARM Cortex-M4-based microcontroller and (2) *liboqs* library for the Raspberry Pi 3 Model B Plus based on the ARM Cortex-A53 processor. Our performance evaluation reveals that the presented custom countermeasures incur reasonable performance overheads on both the evaluated embedded platforms. We therefore believe our work argues for usage of custom countermeasures within real-world implementations of lattice-based schemes, either in a standalone manner or as reinforcements to generic countermeasures such as masking.

³We do not report results for the Verify_Loop_Abort and Verify_Add (Sign countermeasure on the ARM Cortex-A53 device in the article, but we note that the countermeasures implemented on the ARM Cortex-M4 device in C language can be easily ported to the ARM Cortex-A53 device.

ACKNOWLEDGEMENT

We would like to thank Dr. Sujoy Sinha Roy from TU Graz for very useful discussions regarding categorization of side-channel and fault attacks on lattice-based schemes.

REFERENCES

- [1] Gorjan Alagic, Jacob Alperin-Sheriff, Daniel Apon, David Cooper, Quynh Dang, John Kelsey, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta et al. 2020. Status report on the second round of the NIST post-quantum cryptography standardization process. *US Department of Commerce, NIST* (2020). <https://www.nist.gov/publications/status-report-second-round-nist-post-quantum-cryptography-standardization-process>
- [2] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Carl Miller, Dustin Moody, Rene Peralta et al. 2022. Status report on the third round of the NIST post-quantum cryptography standardization process. *National Institute of Standards and Technology* (2022).
- [3] Dorian Amiet, Andreas Curiger, Lukas Leuenberger, and Paul Zbinden. 2020. Defeating NewHope with a single trace. In *International Conference on Post-quantum Cryptography*. Springer, 189–205.
- [4] Daniel Apon and James Howe. 2021. Attacks on NIST PQC 3rd round candidates. Invited talk at Real World Crypto 2021, Retrieved from <https://iacr.org/submit/files/slides/2021/rwc/rwc2021/22/slides.pdf>.
- [5] Roberto Avanzi, Joppe W. Bos, Leo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2021. CRYSTALS-Kyber (version 3.02): Algorithm specifications and supporting documentation. Retrieved from <https://pq-crystals.org/kyber/data/kyber-specification-round3.pdf>.
- [6] Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Markus Schönauer, Tobias Schneider, François-Xavier Standaert, and Christine van Vredendaal. 2022. Leveling Dilithium against Leakage: Revisited Sensitivity Analysis and Improved Implementations. *Cryptology ePrint Archive* (2022). <https://src.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization-conference/documents/papers/leveling-dilithium-against-leakage-pqc2022.pdf>
- [7] Florian Bache, Clara Paglialonga, Tobias Oder, Tobias Schneider, and Tim Güneysu. 2020. High-speed masking for polynomial comparison in lattice-based KEMs. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 3 (2020), 483–507. DOI : <https://doi.org/10.13154/tches.v2020.i3.483-507>
- [8] Ciprian Baetu, F. Betül Durak, Loïs Huguénin-Dumittan, Abdullah Talayhan, and Serge Vaudenay. 2019. Misuse attacks on post-quantum cryptosystems. In *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part II (Lecture Notes in Computer Science)*, Yuval Ishai and Vincent Rijmen (Eds.), Vol. 11477. Springer, 747–776. DOI : https://doi.org/10.1007/978-3-030-17656-3_26
- [9] Hanno Becker, Vincent Hwang, Matthias J. Kannwischer, Bo-Yin Yang, and Shang-Yi Yang. 2022. Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2022, 1 (2022), 221–244. <https://eprint.iacr.org/2021/986>
- [10] Michiel Van Beirendonck, Jan-Pieter D’anvers, Angshuman Karmakar, Josep Balasch, and Ingrid Verbauwhede. 2021. A side-channel-resistant implementation of SABER. *ACM J. Emerg. Technol. Comput. Syst.* 17, 2 (2021), 1–26.
- [11] Shivam Bhasin, Jan-Pieter D’Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel van Beirendonck. 2021. Attacking and defending masked polynomial comparison for lattice-based cryptography. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 3 (2021), 334–359. DOI : <https://doi.org/10.46586/tches.v2021.i3.334-359>
- [12] Nina Bindel, Johannes Buchmann, and Juliane Krämer. 2016. Lattice-based signature schemes and their sensitivity to fault attacks. In *Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE Computer Society, 63–77. DOI : <https://doi.org/10.1109/FDTC.2016.11>
- [13] Joppe W. Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. 2021. Masking Kyber: First- and higher-order implementations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 4 (2021), 173–214. DOI : <https://doi.org/10.46586/tches.v2021.i4.173-214>
- [14] Leon Botros, Matthias J. Kannwischer, and Peter Schwabe. 2019. Memory-efficient high-speed implementation of Kyber on Cortex-M4. In *Progress in Cryptology - AFRICACRYPT 2019 - 11th International Conference on Cryptology in Africa, Rabat, Morocco, July 9–11, 2019, Proceedings*. Springer, 209–228. DOI : https://doi.org/10.1007/978-3-030-23696-0_11
- [15] Leon Groot Bruinderink and Peter Pessl. 2018. Differential fault attacks on deterministic lattice signatures. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018, 3 (2018). Retrieved from <https://eprint.iacr.org/2018/355.pdf>.
- [16] Zhaohui Chen, Emre Karabulut, Aydin Aysu, Yuan Ma, and Jiwu Jing. 2021. An efficient non-profiled side-channel attack on the CRYSTALS-Dilithium post-quantum signature. In *39th IEEE International Conference on Computer Design, ICCD 2021, Storrs, CT, USA, October 24–27, 2021*. IEEE, 583–590. DOI : <https://doi.org/10.1109/ICCD53106.2021.00094>

- [17] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. 2020. LWE with side information: Attacks and concrete security estimation. In *Advances in Cryptology – CRYPTO 2020*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer International Publishing, Cham, 329–358.
- [18] Jan-Pieter D’Anvers, Daniel Heinz, Peter Pessl, Michiel Van Beirendonck, and Ingrid Verbauwhede. 2022. Higher-order masked ciphertext comparison for lattice-based cryptography. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2022, 2 (2022), 115–139. DOI : <https://doi.org/10.46586/tches.v2022.i2.115-139>
- [19] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. Decryption failure attacks on IND-CCA secure lattice-based schemes. In *Public-Key Cryptography – PKC 2019*, Dongdai Lin and Kazue Sako (Eds.). Springer International Publishing, Cham, 565–598.
- [20] Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. 2020. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In *Advances in Cryptology – EUROCRYPT 2020*, Anne Canteaut and Yuval Ishai (Eds.). Springer International Publishing, Cham, 3–33.
- [21] Jan-Pieter D’Anvers, Marcel Tiepelt, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. Timing attacks on error correcting codes in post-quantum schemes. In *ACM Workshop on Theory of Implementation Security*. 2–9.
- [22] Jeroen Delvaux. 2021. Roulette: Breaking Kyber with diverse fault injection setups. *Cryptology ePrint Archive* (2021), 1622.
- [23] Léo Ducas, Alain Durmus, Tancrede Lepoint, and Vadim Lyubashevsky. 2013. Lattice signatures and bimodal Gaussians. In *Advances in Cryptology – CRYPTO 2013 – 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2013. Proceedings, Part I (Lecture Notes in Computer Science)*, Ran Canetti and Juan A. Garay (Eds.), Vol. 8042. Springer, 40–56. DOI : https://doi.org/10.1007/978-3-642-40041-4_3
- [24] Léo Ducas, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. Crystals-Dilithium: Digital signatures from module lattices. Retrieved from <https://pq-crystals.org/dilithium/data/dilithium-specification-round3.pdf>.
- [25] Thomas Espitau, Pierre-Alain Fouque, Benoît Gérard, and Mehdi Tibouchi. 2016. Loop-abort faults on lattice-based Fiat-Shamir and hash-and-sign signatures. In *International Conference on Selected Areas in Cryptography*. Springer, 140–158.
- [26] Eiichiro Fujisaki and Tatsuaki Okamoto. 1999. Secure integration of asymmetric and symmetric encryption schemes. In *Annual International Cryptology Conference*. Springer, 537–554.
- [27] Denisa O. C. Greconici, Matthias J. Kannwischer, and Daan Sprenkels. 2021. Compact Dilithium implementations on Cortex-M3 and Cortex-M4. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 1 (2021), 1–24.
- [28] Tim Güneysu, Vadim Lyubashevsky, and Thomas Pöppelmann. 2012. Practical lattice-based cryptography: A signature scheme for embedded systems. In *International Conference on Cryptographic Hardware and Embedded Systems*. Springer, 530–547.
- [29] Qian Guo, Thomas Johansson, and Alexander Nilsson. 2020. A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In *Annual International Cryptology Conference*. Springer, 359–386.
- [30] Mike Hamburg, Julius Hermelink, Robert Primas, Simona Samardjiska, Thomas Schamberger, Silvan Streit, Emanuele Strieder, and Christine van Vredendaal. 2021. Chosen ciphertext k-trace attacks on masked CCA2 secure Kyber. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 4 (2021), 88–113.
- [31] Jaeseung Han, Taeho Lee, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Jihoon Cho, Dong-Guk Han, and Bo-Yeon Sim. 2021. Single-trace attack on NIST round 3 candidate Dilithium using machine learning-based profiling. *IEEE Access* 9 (2021), 166283–166292. DOI : <https://doi.org/10.1109/ACCESS.2021.3135600>
- [32] Daniel Heinz, Matthias J. Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Daan Sprenkels. 2022. First-order masked Kyber on ARM Cortex-M4. *IACR Cryptol. ePrint Arch.* (2022), 58. Retrieved from <https://eprint.iacr.org/2022/058>.
- [33] Julius Hermelink, Peter Pessl, and Thomas Pöppelmann. 2021. Fault-enabled chosen-ciphertext attacks on Kyber. In *International Conference on Cryptology in India*. Springer, 311–334.
- [34] Julius Hermelink, Silvan Streit, Emanuele Strieder, and Katharina Thieme. 2022. Adapting belief propagation to counter shuffling of NTTs. *IACR Cryptol. ePrint Arch.* (2022), 555. Retrieved from <https://eprint.iacr.org/2022/555>.
- [35] Saad Islam, Koksai Mus, Richa Singh, Patrick Schaumont, and Berk Sunar. 2022. Signature correction attack on Dilithium signature scheme. In *7th IEEE European Symposium on Security and Privacy, EuroS&P 2022, Genoa, Italy, June 6–10, 2022*. IEEE, 647–663. DOI : <https://doi.org/10.1109/EuroSP53844.2022.00046>
- [36] Matthias J. Kannwischer, Peter Pessl, and Robert Primas. 2020. Single-trace attacks on Keccak. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 3 (2020), 243–268. DOI : <https://doi.org/10.13154/tches.v2020.i3.243-268>
- [37] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. 2019. PQM4: Post-quantum crypto library for the ARM Cortex-M4. Retrieved from <https://github.com/mupq/pqm4>.

- [38] Yanbin Li, Jiajie Zhu, Yuxin Huang, Zhe Liu, and Ming Tang. 2022. Single-trace side-channel attacks on the Toom-Cook: The case study of Saber. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2022, 4 (2022), 285–310. DOI : <https://doi.org/10.46586/tches.v2022.i4.285-310>
- [39] Xianhui Lu, Yamin Liu, Zhenfei Zhang, Dingding Jia, Haiyang Xue, Jingnan He, and Bao Li. 2018. LAC: Practical ring-LWE based public-key encryption with byte-level modulus. *IACR Cryptol. ePrint Arch.* (2018), 1009. Retrieved from <https://eprint.iacr.org/2018/1009>.
- [40] Vadim Lyubashevsky. 2009. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 598–616.
- [41] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2013. On ideal lattices and learning with errors over rings. *J. ACM* 60, 6 (2013), 43.
- [42] Soundes Marzougui, Vincent Ulitzsch, Mehdi Tibouchi, and Jean-Pierre Seifert. 2022. Profiling side-channel attacks on Dilithium: A small bit-fiddling leak breaks it all. *IACR Cryptol. ePrint Arch.* (2022), 106. Retrieved from <https://eprint.iacr.org/2022/106>.
- [43] Vincent Migliore, Benoît Gérard, Mehdi Tibouchi, and Pierre-Alain Fouque. 2019. Masking Dilithium—Efficient implementation and side-channel evaluation. In *Applied Cryptography and Network Security - 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings (Lecture Notes in Computer Science)*, Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung (Eds.), Vol. 11464. Springer, 344–362. DOI : https://doi.org/10.1007/978-3-030-21568-2_17
- [44] Catinca Muijdei, Arthur Beckers, Jose Bermundo, Angshuman Karmakar, Lennert Wouters, and Ingrid Verbauwhede. 2022. Side-channel analysis of lattice-based post-quantum cryptography: Exploiting polynomial multiplication. *IACR Cryptol. ePrint Arch.* (2022), 474. DOI : <https://eprint.iacr.org/2022/474>
- [45] Hamid Nejatollahi, Nikil Dutt, Sandip Ray, Francesco Regazzoni, Indranil Banerjee, and Rosario Cammarota. 2019. Post-quantum lattice-based cryptography implementations: A survey. *ACM Comput. Surv.* 51, 6 (2019), 1–41.
- [46] Kalle Ngo, Elena Dubrova, Qian Guo, and Thomas Johansson. 2021. A side-channel attack on a masked IND-CCA secure Saber KEM implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 4 (2021), 676–707.
- [47] Kalle Ngo, Elena Dubrova, and Thomas Johansson. 2021. Breaking masked and shuffled CCA secure Saber KEM by power analysis. In *5th Workshop on Attacks and Solutions in Hardware Security*. 51–61.
- [48] Kalle Ngo, Ruize Wang, Elena Dubrova, and Nils Paulsru. 2022. Side-channel attacks on lattice-based KEMs are not prevented by higher-order masking. *IACR Cryptol. ePrint Arch.* (2022), 919. Retrieved from <https://eprint.iacr.org/2022/919>.
- [49] Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. 2018. Practical CCA2-secure and masked ring-LWE implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018, 1 (2018), 142–174.
- [50] Judea Pearl. 1986. Fusion, propagation, and structuring in belief networks. *Artif. Intell.* 29, 3 (1986), 241–288.
- [51] Peter Pessl and Robert Primas. 2019. More practical single-trace attacks on the number theoretic transform. In *International Conference on Cryptology and Information Security in Latin America*. Springer, 130–149.
- [52] Peter Pessl and Lukas Prokop. 2021. Fault attacks on CCA-secure lattice KEMs. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 2 (2021), 37–60.
- [53] Robert Primas, Peter Pessl, and Stefan Mangard. 2017. Single-trace side-channel attacks on masked lattice-based encryption. In *International Conference on Cryptographic Hardware and Embedded Systems*. Springer, 513–533.
- [54] Yue Qin, Chi Cheng, Xiaohan Zhang, Yanbin Pan, Lei Hu, and Jintai Ding. 2021. A systematic approach and analysis of key mismatch attacks on lattice-based NIST candidate KEMs. In *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV (Lecture Notes in Computer Science)*, Mehdi Tibouchi and Huaxiong Wang (Eds.), Vol. 13093. Springer, 92–121. DOI : https://doi.org/10.1007/978-3-030-92068-5_4
- [55] Gokulnath Rajendran, Prasanna Ravi, Jan-Pieter D’Anvers, Shivam Bhasin, and Anupam Chattopadhyay. 2022. Pushing the limits of generic side-channel attacks on LWE-based KEMs—Parallel PC Oracle attacks on Kyber KEM and beyond. *IACR Cryptol. ePrint Arch.* (2022), 931. Retrieved from <https://eprint.iacr.org/2022/931>.
- [56] P. Ravi, S. Bhasin, S. S. Roy, and A. Chattopadhyay. 2022. On exploiting message leakage in (Few) NIST PQC candidates for practical message recovery attacks. In *IEEE Transactions on Information Forensics and Security* 17 (2022), 684–699. DOI : [10.1109/TIFS.2021.3139268](https://doi.org/10.1109/TIFS.2021.3139268)
- [57] Prasanna Ravi, Sourav Sen Gupta, Anupam Chattopadhyay, and Shivam Bhasin. 2019. Improving speed of Dilithium’s signing procedure. In *International Conference on Smart Card Research and Advanced Applications*. Springer, 57–73.
- [58] Prasanna Ravi, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. 2021. Lattice-based key-sharing schemes: A survey. *ACM Comput. Surv.* 54, 1 (2021), 1–39.
- [59] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. 2018. Side-channel assisted existential forgery attack on Dilithium—a NIST PQC candidate. *Cryptology ePrint Archive* (2018).

- [60] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. 2019. Exploiting determinism in lattice-based signatures: Practical fault attacks on pqm4 implementations of NIST candidates. In *ACM Asia Conference on Computer and Communications Security*. 427–440.
- [61] Prasanna Ravi, Romain Poussier, Shivam Bhasin, and Anupam Chattopadhyay. 2020. On configurable SCA countermeasures against single trace attacks for the NTT. In *International Conference on Security, Privacy, and Applied Cryptography Engineering*. Springer, 123–146.
- [62] Prasanna Ravi, Debapriya Basu Roy, Shivam Bhasin, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. 2019. Number “not used” once-practical fault attack on pqm4 implementations of NIST candidates. In *International Workshop on Constructive Side-channel Analysis and Secure Design*. Springer, 232–250.
- [63] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. 2020. Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 3 (2020), 307–335.
- [64] Prasanna Ravi, Bolin Yang, Shivam Bhasin, Fan Zhang, and Anupam Chattopadhyay. 2022. Fiddling the twiddle constants—Fault injection analysis of the number theoretic transform. *IACR Cryptol. ePrint Arch.* (2022), 824. Retrieved from <https://eprint.iacr.org/2022/824>.
- [65] Oded Regev. 2009. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM* 56, 6 (2009), 1–40.
- [66] Muyan Shen, Chi Cheng, Xiaohan Zhang, Qian Guo, and Tao Jiang. 2022. Find the bad apples: An efficient method for perfect key recovery under imperfect SCA oracles—A case study of Kyber. *IACR Cryptol. ePrint Arch.* (2022), 563. Retrieved from <https://eprint.iacr.org/2022/563>.
- [67] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Tae-Ho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. 2020. Single-trace attacks on message encoding in lattice-based KEMs. 8 (2020), 183175–183191.
- [68] Douglas Stebila and Michele Mosca. 2016. Post-quantum key exchange for the internet and the open quantum safe project. In *International Conference on Selected Areas in Cryptography*. Springer, 14–37.
- [69] Hauke Steffen, Georg Land, Lucie Kogelheide, and Tim Güneysu. 2022. Breaking and protecting the crystal: Side-channel analysis of Dilithium in hardware. *Cryptology ePrint Archive* (2022).
- [70] Yutaro Tanaka, Rei Ueno, Keita Xagawa, Akira Ito, Junko Takahashi, and Naofumi Homma. 2022. Multiple-valued plaintext-checking side-channel attacks on post-quantum KEMs. *IACR Cryptol. ePrint Arch.* (2022), 940. Retrieved from <https://eprint.iacr.org/2022/940>.
- [71] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. 2022. Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2022, 1 (2022), 296–322. <https://doi.org/10.46586/tches.v2022.i1.296-322>
- [72] Nicolas Veyrat-Charvillon, Benoît Gérard, and François-Xavier Standaert. 2014. Soft analytical side-channel attacks. In *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014. Proceedings, Part I (Lecture Notes in Computer Science)*, Palash Sarkar and Tetsu Iwata (Eds.), Vol. 8873. Springer, 282–296. https://doi.org/10.1007/978-3-662-45611-8_15
- [73] Ruize Wang, Kalle Ngo, and Elena Dubrova. 2022. Making biased DL models work: Message and key recovery attacks on saber using amplitude-modulated EM emanations. *IACR Cryptol. ePrint Arch.* (2022), 852. <https://eprint.iacr.org/2022/852>.
- [74] Ruize Wang, Kalle Ngo, and Elena Dubrova. 2022. Side-channel analysis of Saber KEM using amplitude-modulated EM emanations. *IACR Cryptol. ePrint Arch.* (2022), 807. <https://eprint.iacr.org/2022/807>.
- [75] Keita Xagawa, Akira Ito, Rei Ueno, Junko Takahashi, and Naofumi Homma. 2021. Fault-injection attacks against NIST’s post-quantum cryptography round 3 KEM candidates. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 33–61.
- [76] Zhuang Xu, Owen Michael Pemberton, Sujoy Sinha Roy, David Oswald, Wang Yao, and Zhiming Zheng. 2021. Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of Kyber. *IEEE Trans. Comput.* (2021).

Received 10 May 2022; revised 28 February 2023; accepted 7 May 2023